Neural Networks 201 (2026) 108917

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Maximum total correntropy-based broad learning system with robust M-estimator

Tao Chen, Xi Chen, Wei Li *

School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, Hubei, 430074, China

ARTICLE INFO

Keywords:
Broad learning system (BLS)
Maximum total correntropy criterion
M-estimator function
Fixed-point iteration

ABSTRACT

Although the broad learning system (BLS) and its existing robust variants have been widely applied in various fields due to their excellent performance, they still cannot effectively handle noise present in the input data of training samples, which may lead to a decline in algorithm performance. To address this issue, this paper proposes a maximum total correntropy-based BLS (MTC-BLS), which trains the model using the maximum total correntropy (MTC) criterion. This endows the model with the advantages of both the total least squares (TLS) method and the MCC criterion, enabling it to effectively handle both input and output noise. Furthermore, to mitigate the impact of kernel width selection deviations in the entropy-based criterion, two new robust methods are further proposed, namely MMTC-BLSa and MMTC-BLSb. The former directly incorporates the M-estimator into the algorithm's objective function, using the dual constraints of the M-estimator and MTC to balance model predictive performance while reducing the algorithm's dependence on the kernel width. The latter uses the M-estimator as a weighting factor in the MTC criterion for model training, employing the M-estimator to compensate for model errors caused by kernel width deviations, thereby effectively mitigating the negative effects of such deviations. Moreover, to enable efficient training, fixed-point iteration methods are introduced to provide iterative solution strategies for MTC-BLS, MMTC-BLSa, and MMTC-BLSb, respectively. The effectiveness and superiority of the proposed methods are validated through multiple comparative experiments on time series datasets, regression datasets and image datasets.

1. Introduction

In recent years, deep neural networks (DNNs) have exhibited remarkable capabilities in feature extraction and nonlinear prediction across various tasks. Nevertheless, with the exponential growth of data, deep networks, by continuously increasing the number of layers and employing complex weight iteration methods, inevitably lead to issues such as diminished training speed (Rasch et al., 2024) and gradient explosion (Ceni, 2025). To address these issues, Chen and Liu (2018) proposed a novel and highly efficient lightweight neural network called the broad learning system (BLS). BLS uses random mapping to extract feature nodes from the input data, and enhances these feature nodes through nonlinear mapping to obtain enhancement nodes, which are then integrated to form the final hidden layer of the network. Clearly, compared to DNNs, the network construction process and architecture of BLS are both comparatively more concise. In addition, the weights of the hidden layer are randomly generated and fixed in the training process, requiring only the use of a pseudo-inverse operation to quickly obtain the connection weights between the output layer and the hidden layer, which further significantly reduces the training cost of the algorithm. With these unique advantages, BLS has now been widely applied in fields such as fault diagnosis (Qin et al., 2025; Xu et al., 2024), visual classification (Mallea et al., 2026; Zhao et al., 2022), and regression prediction (Peng & ChunHao, 2022; Yuen & Kuok, 2025).

Nevertheless, BLS employs the minimum mean square error criterion (MMSE) to construct the objective function, enabling fast training speed, but this simultaneously restricts the model's applicability, allowing it to perform excellently only in noise-free or Gaussian noise environments. To this end, several more robust cost functions are constructed in BLS. Zhang et al. (2022) proposed an error entropy-based BLS (BLS-QMEE), which constructs the objective function of BLS by introducing the quantized minimum error entropy criterion (QMEE), thereby enabling the model to effectively mitigate the adverse effects of outliers. However, due to the limitations of QMEE, the performance of BLS-QMEE deteriorates when handling zero-mean errors. In contrast to BLS-QMEE, which uses the entropy criterion, robust BLS (RBLS) (Guo & Xu, 2023) constructs a weighting operator by introducing M-estimators, thereby reducing the weight of outlier samples and improving the model's

* Corresponding author.
E-mail addresses: qdu.chentao@qdu.edu.cn (T. Chen), chenxi@hust.edu.cn (X. Chen), liwei0828@mail.hust.edu.cn (W. Li).

https://doi.org/10.1016/j.neunet.2026.108917
Received 24 November 2025; Received in revised form 16 February 2026; Accepted 28 March 2026
Available online 31 March 2026

prediction performance. Note that, when the training samples contain extreme outliers, RBLS is highly susceptible to the dominance of these outliers, leading to a deterioration in performance. Besides introducing the error entropy criterion, multiple BLS variants based on the maximum correntropy criterion (Wang et al., 2026; Zhao & Lu, 2024; Zhao et al., 2025; Zheng et al., 2021, 2024) have also been proposed. Correntropy, as a kernel function that measures the generalized similarity between random variables, can characterize the higher-order moments of the errors to enhance the robustness of BLS. However, existing robust BLS models that incorporate various correntropy variants are not applicable to situations where the input data contains noise, and cannot effectively handle noise in input. Additionally, when the hyperparameters of the kernel function used in the maximum correntropy criterion (MCC) deviate from their optimal values (either too large or too small), it may lead to a deterioration in the algorithm's performance.

The entropy-based methods from information theory (Polyanskiy & Wu, 2025; Yuan et al., 2024) can effectively enhance the robustness of BLS. Considering that the existing maximum correntropy-based BLS cannot effectively handle noise in the input data, this paper proposes a novel robust BLS (MTC-BLS) based on the maximum total correntropy criterion (MTC) (He et al., 2019; Hou et al., 2022; Wang et al., 2017). MTC-BLS combines the advantages of the total least squares (TLS) method and the MCC criterion, effectively handling noise in both input and output data. As a result, MTC-BLS can extract data features more efficiently. Furthermore, to reduce the dependence of MTC-BLS on the selection of kernel parameters in the maximum correntropy criterion, we propose two categories of new robust methods, called MMTC-BLSa and MMTC-BLSb. The former directly incorporates the M-estimator as part of the objective function for model training. By simultaneously utilizing the M-estimator and MTC as dual constraints, it ensures the model's robustness, thereby reducing the algorithm's reliance on kernel parameter selection. The latter uses the M-estimator as a weighting coefficient within MTC criterion to train the model. By employing the M-estimator to compensate for the model errors caused by biases in kernel parameter selection, it effectively suppresses the adverse effects of kernel parameter deviations from their optimal values. Finally, multiple robustness experiments demonstrate the superior robustness of the three types of BLS variants proposed in this paper. In summary, the main contributions of this paper are listed as follows.

1) By reconstructing the objective function of BLS based on the MTC criterion, MTC-BLS is obtained. It combines the advantages of the TLS method and the MCC criterion, enabling more efficient feature extraction while effectively handling noise in both input and output data.

2) Two categories of new robustness BLS methods (i.e., MMTC-BLSa and MMTC-BLSb) are proposed to reduce the dependency of MTC-BLS on kernel parameter selection. MMTC-BLSa guarantees the model robustness by simultaneously utilizing the M-estimator function and the MTC criterion as dual constraints, and MMTC-BLSb adjusts the model errors caused by biases in kernel parameter selection by using the M-estimator as a weighting coefficient for MTC criterion.

3) By introducing the fixed-point iteration method as an optimization theory, iterative solution strategies for MTC-BLS, MMTC-BLSa, and MMTC-BLSb are presented. Furthermore, the convergence property and computational complexity of the iteration are also analyzed.

4) Robustness experiments on the regression datasets and time series datasets validate the robustness of the three types of novel RBLS. Meanwhile, classification experiments also demonstrate the effectiveness of the new algorithms.

2. Preliminary knowledge

This section mainly provides a detailed introduction to the basic concepts of the broad learning system and the maximum total correlation entropy, which will help readers understand our main work.

2.1. Broad learning system (BLS)

As a new type of flat neural network, BLS has demonstrated outstanding performance in various fields, and its network architecture is shown in Fig. 1. It can be easily observed that BLS consists of an input layer, a feature layer, an enhancement layer and an output layer. Among them, the feature layer and the enhancement layer together form the hidden layer of BLS. For a training dataset $\{(\boldsymbol{X},\boldsymbol{Y}) \mid \boldsymbol{X}\in\mathbb{R}^{N\times C},\boldsymbol{Y}\in\mathbb{R}^{N}\}$ with N samples and C-dimensional input features, the pth feature node group in the feature layer of BLS can be formulated as

$$\boldsymbol{F}_p = \psi\left(\boldsymbol{X}\boldsymbol{W}_{F_p} + \boldsymbol{b}_{F_p}\right),\ p = 1,2,\ldots,P \tag{1}$$

where $\psi(\cdot)$ is an activation function, $\boldsymbol{W}_{F_p}$ and $\boldsymbol{b}_{F_p}$ are the weights and the biases that randomly generated by a sparse auto-encoder. Then, the feature can be denoted as $\boldsymbol{F}^P = [\boldsymbol{F}_1,\boldsymbol{F}_2,\ldots,\boldsymbol{F}_P]$, and the qth enhancement node group in the enhancement layer can be formulated as

$$\boldsymbol{E}_q = \phi\left(\boldsymbol{F}^P\boldsymbol{W}_{E_q} + \boldsymbol{b}_{E_q}\right),\ q = 1,2,\ldots,Q \tag{2}$$

where $\phi(\cdot)$ is another activation function, $\boldsymbol{W}_{E_q}$ and $\boldsymbol{b}_{E_q}$ are also the weights and the biases that randomly generated by sparse auto-encoder. The obtained $\boldsymbol{E}_1,\boldsymbol{E}_2,\ldots,\boldsymbol{E}_Q$ can be connected as the enhancement layer, that is, $\boldsymbol{E}^Q = [\boldsymbol{E}_1,\boldsymbol{E}_2,\ldots,\boldsymbol{E}_Q]$.

By concatenating $\boldsymbol{F}^P$ and $\boldsymbol{E}^Q$ to form the hidden layer $\boldsymbol{A} = [\boldsymbol{F}^P,\boldsymbol{E}^Q]$, the output results of BLS can be presented as $\hat{\boldsymbol{Y}} = \boldsymbol{A}\boldsymbol{W}$. Then the final weight vector $\boldsymbol{W}$ can be obtained by minimizing the following objective function:

$$\arg\min_{\boldsymbol{W}}\ \left(\|\boldsymbol{Y} - \boldsymbol{A}\boldsymbol{W}\|_2^2 + \lambda\|\boldsymbol{W}\|_2^2\right) \tag{3}$$

where λ denotes the regularization coefficient. Then, the ridge regression method is used to solve Eq. (3), that is,

$$\boldsymbol{W} = \lim_{\lambda\to 0}\left(\lambda\boldsymbol{I} + \boldsymbol{A}^T\boldsymbol{A}\right)^{-1}\boldsymbol{A}^T\boldsymbol{Y} \tag{4}$$

where I is a unit matrix.

2.2. Maximum total correntropy (MTC)

The maximum correntropy criterion (Principe, 2010) is a robust optimization criterion grounded in information theory learning. Its core idea is to measure the similarity between the actual output sequence $\hat{\boldsymbol{Y}}$ and the expected output sequence $\boldsymbol{Y}$ of the model in the reproducing kernel Hilbert space (RKHS) using the correntropy, and then to determine the model parameters by maximizing this similarity. For the actual sequence $\hat{\boldsymbol{Y}}$ and the expected sequence $\boldsymbol{Y}$, the correntropy can be defined in the kernel space as

$$\mathcal{V}_\sigma\left(\hat{\boldsymbol{Y}} - \boldsymbol{Y}\right) = E\left[\kappa_\sigma\left(\hat{\boldsymbol{Y}} - \boldsymbol{Y}\right)\right] \tag{5}$$

where $E[\cdot]$ is an expectation operator. And κ_σ represents the Mercer kernel controlled by the kernel width σ, and it is usually selected as the Gaussian kernel function, *i.e.*, $\kappa_\sigma(\hat{y}-y) = \frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{(\hat{y}-y)}{2\sigma^2}\right)$.

It should be noted that when using maximum relevance entropy, it is necessary to effectively handle the input noise of the training data; otherwise, it cannot be well applied to actual prediction tasks. Therefore, Wang et al. (2017) proposed the maximum total correntropy criterion by introducing a normalized weighted error, which can effectively handle noise in both input and output data. The criterion can be expressed as

$$\begin{aligned}\mathcal{V}_\sigma\left(\hat{\boldsymbol{Y}} - \boldsymbol{Y}\right) &= E\left[\kappa_\sigma\left(\frac{\hat{\boldsymbol{Y}} - \boldsymbol{Y}}{\sqrt{\bar{\boldsymbol{W}}^T\bar{\boldsymbol{W}}}}\right)\right] \\ &= E\left[\exp\left(-\frac{\left\|\hat{\boldsymbol{Y}} - \boldsymbol{Y}\right\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right)\right]\end{aligned} \tag{6}$$

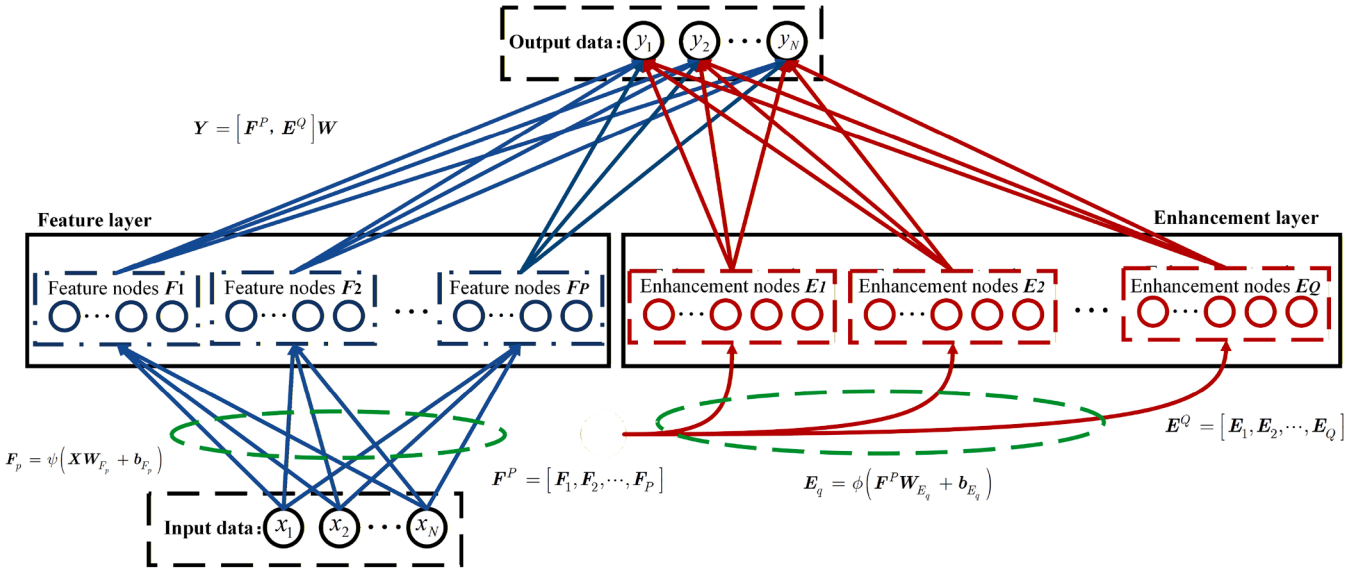


Fig. 1. The network architecture of BLS.

where $\bar{\boldsymbol{W}} = \begin{bmatrix} \sqrt{\frac{\sigma_o^2}{\sigma_i^2}} & -\boldsymbol{W}^T \end{bmatrix}^T$ denotes the augmented weighting vector, and σ_i^2 and σ_o^2 represent the input and output noise variances, respectively. Considering that the noise distribution in actual tasks is unknown, and the training sample size is limited, then Eq. (6) can be approximated as

$$\mathcal{V}_\sigma\left(\hat{\boldsymbol{Y}} - \boldsymbol{Y}\right) = \frac{1}{N}\sum_{i=1}^{N} \exp\left(-\frac{\left\|\hat{\boldsymbol{Y}}_i - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) \tag{7}$$

Similar to correntropy, with the objective of minimizing the error between the actual output and the desired output, we achieve this by maximizing the total correntropy, *i.e.*,

$$\mathcal{J} = \arg\max_{\boldsymbol{W}} \frac{1}{N}\sum_{i=1}^{N} \exp\left(-\frac{\left\|\hat{\boldsymbol{Y}}_i - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) \tag{8}$$

this optimization criterion is referred to as the maximum total correntropy criterion.

3. Proposed algorithms

To address the issue that BLS and some of its existing variants primarily rely on the ridge regression for model training, making them highly sensitive to noise and outliers, this section introduces three improved algorithms by integrating the maximum total correntropy criterion and robust M-estimators, namely MTC-BLS, MMTC-BLSa, and MMTC-BLSb. The former optimizes the model by introducing the maximum total correntropy criterion to enhance its ability to handle noise and outliers in input data. Considering that deviations of the kernel width in the entropy criterion from its optimal value (either excessively large or small) can lead to performance fluctuations or even degradation, the latter two algorithms further incorporate M-estimators to suppress the divergence tendency caused by kernel width bias, thereby enhancing the robustness of the model. In the following, we present a detailed introduction to the three proposed methods.

3.1. MTC-BLS

To preserve the structural advantages of BLS, we keep the node generation method unchanged and consider introducing maximum total correntropy to optimize the output weight parameters of the model, instead of directly solving them using ridge regression. Then the objective function of MTC-BLS is defined as

$$\mathcal{J}_M = \arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \exp\left(-\frac{\left\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) - \frac{\lambda}{2}\|\boldsymbol{W}\|_2^2 \tag{9}$$

where λ is the regularization parameter, and N represents the number of samples. Its first term is the MTC criterion, which measures the similarity between the model prediction $\hat{\boldsymbol{Y}}_i = \boldsymbol{A}_i\boldsymbol{W}$ and the ground truth $\boldsymbol{Y}_i$; the second term is a regularization term, used to prevent model overfitting.

Then, one has the gradient of $\mathcal{J}_M$ with respect to $\boldsymbol{W}$ as follows:

$$\frac{\partial \mathcal{J}_M}{\partial \boldsymbol{W}} = -\frac{1}{\sigma^2}\sum_{i=1}^{N} \exp\left(-\frac{\left\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) \cdot \frac{\boldsymbol{A}_i^T\|\bar{\boldsymbol{W}}\|_2^2(\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i) - \|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\|_2^2\boldsymbol{W}}{\|\bar{\boldsymbol{W}}\|_2^4} - \lambda\boldsymbol{W} \tag{10}$$

By rewriting Eq. (10) in matrix form, one obtains

$$\frac{\partial \mathcal{J}_M}{\partial \boldsymbol{W}} = -\frac{1}{\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M(\boldsymbol{A}\boldsymbol{W} - \boldsymbol{Y}) - s_M\boldsymbol{W}\right) - \lambda\boldsymbol{W} \tag{11}$$

where

$$\boldsymbol{\Lambda}_M = \begin{pmatrix} \exp\left(-\frac{\|\boldsymbol{A}_1\boldsymbol{W} - \boldsymbol{Y}_1\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) & & \\ & \ddots & \\ & & \exp\left(-\frac{\|\boldsymbol{A}_N\boldsymbol{W} - \boldsymbol{Y}_N\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) \end{pmatrix}$$

$$s_M = \sum_{i=1}^{N} \frac{\left\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{\|\bar{\boldsymbol{W}}\|_2^2} \exp\left(-\frac{\left\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right)$$

By setting Eq. (11) to zero, we obtain the weight expression of MTC-BLS as

$$\boldsymbol{W} = \left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A} + \gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y} \tag{12}$$

where $\gamma_M = \lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2 - s_M$. In fact, since both $\boldsymbol{\Lambda}_M$ and γ_M are functions of $\boldsymbol{W}$, Eq. (12) is essentially a fixed-point equation. Therefore, we may assume that

$$h(\boldsymbol{W}) = \left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A} + \gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y} \tag{13}$$

Algorithm 1 MTC-BLS.

Input: Training data $\boldsymbol{X}$ and $\boldsymbol{Y}$, regularization parameters λ, kernel width σ, input and output noise variances σ_i^2 and σ_o^2, maximum iteration number $t_{\max}$ and iteration tolerance κ.
Output: Output weight $\boldsymbol{W}$.
Initialization: $\boldsymbol{W}_{(0)}$, and iterations $t = 1$.
1. Generate feature layer ($\boldsymbol{F}^P$) by Eq. (1);
2. Generate enhancement layer ($\boldsymbol{E}^Q$) by Eq. (2);
3. Calculate hidden layer $\boldsymbol{A} = \left[\boldsymbol{F}^P, \boldsymbol{E}^Q\right]$;
While $\left\|\boldsymbol{W}_{(t+1)} - \boldsymbol{W}_{(t)}\right\|_2^2 > \kappa$ and $t < t_{\max}$ **do**
4. Update $\bar{\boldsymbol{W}}$ by $\bar{\boldsymbol{W}} = \left[\sqrt{\frac{\sigma_o^2}{\sigma_i^2}} \quad -\boldsymbol{W}^T\right]^T$;
5. Calculate $\boldsymbol{\Lambda}_M$;
6. Calculate s_M and γ_M;
7. Update $\boldsymbol{W}_{(t)}$ by Eq. (14);
8. $t = t + 1$;
end while
9. Set $\boldsymbol{W} = \boldsymbol{W}_{(t)}$.

Subsequently, following the widely used fixed-point iteration method (Bingham & Hyvarinen, 2000; Hale et al., 2008; Heravi & Hodtani, 2017; Wang et al., 2026; Zhao et al., 2025; Zheng et al., 2021), we iteratively update $\boldsymbol{W}$ by

$$\boldsymbol{W}_{(t)} = h\left(\boldsymbol{W}_{(t-1)}\right) \tag{14}$$

where $\boldsymbol{W}_{(t)}$ denotes the model output weights obtained at the tth iteration.

In our MTC-BLS, to prevent excessive iterations that may lead to increased training costs, we set a maximum number of iterations $t_{\max}$. Furthermore, to determine the efficiency of model training, let κ denote the iteration tolerance, based on which the termination criterion can be set as $\left\|\boldsymbol{W}_{(t)} - \boldsymbol{W}_{(t-1)}\right\|_2^2 < \kappa$, where κ is set as 0.05 in our paper. When the changing trend of $\boldsymbol{W}$ diminishes with the increase of iterations and eventually falls below κ, it indicates that $\boldsymbol{W}$ has gradually converged to a stable state, and further iterations are unnecessary to obtain the final model output weights. Algorithm 1 presents the detailed procedure.

It is worth noting that in this work, the introduction of the MTC criterion is not based on the assumption that the noise in the mapped feature matrix $\boldsymbol{A}$ is additive or Gaussian. In practice, the nonlinear activation functions in BLS inevitably distort the distribution of input noise, which is one of the central challenges in recent studies on robust BLS (Li, 2025; Mallea et al., 2026). The normalized structure $\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\|_2^2 / \|\bar{\boldsymbol{W}}\|_2^2$ is adopted primarily for its regularization effect on the output weights. This idea originates from TLS, but its effectiveness has been theoretically validated within nonlinear regression frameworks (Wang et al., 2024). The introduction of the Gaussian kernel further exploits its ability to robustly suppress large residuals. Extensive studies (Li et al., 2025; Wang et al., 2026) from 2025 to 2026 have demonstrated that correntropy-based criteria maintain their adaptive weighting capability even in feature spaces after nonlinear mappings. Therefore, the effectiveness of MTC-BLS is both empirically verifiable and theoretically justifiable.

3.2. MMTC-BLSa and MMTC-BLSb

In the preceding section, we introduced the theoretical foundation of the proposed MTC-BLS algorithm. Although MTC-BLS can improve prediction performance compared with most existing improved BLS methods, its performance is highly sensitive to the choice of kernel width. To reduce the algorithm's dependence on kernel parameters and further enhance model performance, we construct MMTC-BLS by incorporating M-estimators.

In this paper, two construction strategies for MMTC-BLS are proposed. The first strategy, referred to as MMTC-BLSa, directly incorporates the M-estimator as part of the algorithm's objective function for model training. By simultaneously leveraging the dual constraints of the M-estimator and maximum total correntropy, this approach balances the predictive performance of the model and thereby reduces the algorithm's dependence on kernel width. The second strategy, referred to as MMTC-BLSb, introduces the M-estimator as a weighting coefficient of maximum total correntropy for model training. By employing the M-estimator to adjust model errors caused by deviations in kernel width, this approach effectively suppresses the adverse effects of kernel width bias.

For MMTC-BLSa, the objective function is constructed as follows:

$$\begin{aligned} &\arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \exp\left(-\frac{\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) - \sum_{i=1}^{N}\rho(e_i) - \frac{\lambda}{2}\|\boldsymbol{W}\|_2^2 \\ &s.t.\ e_i = \boldsymbol{Y}_i - \boldsymbol{A}_i\boldsymbol{W} \end{aligned} \tag{15}$$

where, $\rho(\cdot)$ denotes the cost function of the M-estimator. According to Eq. (15), MMTC-BLSa builds upon MTC-BLS by further introducing the M-estimator function to act synergistically with the MTC criterion, pursuing minimization of the prediction error while simultaneously maximizing the similarity between the model predictions and the actual values. In practice, $\rho(\cdot)$ is commonly chosen as a smoother piecewise function. Compared with estimators such as ridge regression and maximum correntropy, its variation trend is relatively milder or even constant, which can effectively suppress the negative effects of noise and outliers on the learning algorithm.

Considering that $\rho(\cdot)$ in Eq. (15) is an abstract function, a staged optimization strategy is proposed for model training. In the first stage, by applying the Lagrange multiplier method, Eq. (15) can be transformed into the following dual optimization problem

$$\begin{aligned} \mathcal{J}_{MA} = &\sum_{i=1}^{N} \exp\left(-\frac{\|\boldsymbol{A}_i\boldsymbol{W} - \boldsymbol{Y}_i\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) - \sum_{i=1}^{N}\rho(e_i) - \frac{\lambda}{2}\|\boldsymbol{W}\|_2^2 \\ &- \sum_{i=1}^{N}\eta_i\left(\boldsymbol{Y}_i - \boldsymbol{A}_i\boldsymbol{W} - e_i\right) \end{aligned} \tag{16}$$

where, η_i denotes the Lagrange multiplier of the ith constraint. By differentiating the above Lagrangian $\mathcal{J}_{MA}$ with respect to $\boldsymbol{W}$, e_i, and η_i and setting the derivatives to zero, we obtain the KKT optimality conditions (Refer to Appendix A) for Eq. (16):

$$\begin{cases} \frac{\partial \mathcal{J}_M}{\partial \boldsymbol{W}} + \sum_{i=1}^{N}\eta_i\boldsymbol{A}_i = 0 \\ \eta_i - \frac{\partial\rho(e_i)}{\partial e_i} = 0 \\ \boldsymbol{Y}_i - \boldsymbol{A}_i\boldsymbol{W} - e_i = 0 \end{cases} \tag{17}$$

Assuming that $\phi\{e_i\} = \frac{\partial\rho(e_i)}{\partial e_i}$ is the gradient of $\rho(e_i)$ with respect to the error e_i, and defining $\vartheta_i = \frac{\phi\{e_i\}}{e_i}$ as the weighting function of the M-estimator for e_i, Eq. (17) can be written in matrix form as

$$\begin{cases} \frac{1}{\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M(\boldsymbol{A}\boldsymbol{W} - \boldsymbol{Y}) - s_M\boldsymbol{W}\right) + \lambda\boldsymbol{W} = \boldsymbol{A}^T\boldsymbol{\eta} \\ \boldsymbol{\eta} = \boldsymbol{\vartheta}\boldsymbol{e} \\ \boldsymbol{Y} - \boldsymbol{A}\boldsymbol{W} = \boldsymbol{e} \end{cases} \tag{18}$$

where $\boldsymbol{e} = [e_1, e_2, \ldots, e_N]$ denotes the error vector, $\boldsymbol{\eta} = [\eta_1, \eta_2, \ldots, \eta_N]$ denotes the vector of Lagrange multipliers, and $\boldsymbol{\vartheta} = \mathrm{diag}(\vartheta_1, \vartheta_2, \ldots, \vartheta_N)$ denotes the weighting matrix of the M-estimator. From Eq. (18), it is easy to obtain that

$$\boldsymbol{W} = \left(\boldsymbol{A}^T\boldsymbol{\Lambda}_{MA}\boldsymbol{A} + \gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\boldsymbol{\Lambda}_{MA}\boldsymbol{Y} \tag{19}$$

where $\boldsymbol{\Lambda}_{MA} = \boldsymbol{\Lambda}_M + \sigma^2\|\bar{\boldsymbol{W}}\|_2^2\boldsymbol{\vartheta}$. Evidently, Eq. (19) is also a fixed-point equation. Therefore, in the second stage, the fixed-point iteration method is employed to optimize and solve the output weights $\boldsymbol{W}$. Then,

Algorithm 2 MMTC-BLSa.

Input: Training data $\boldsymbol{X}$ and $\boldsymbol{Y}$, regularization parameters λ, kernel width σ, input and output noise variances σ_i^2 and σ_o^2, maximum iteration number $t_{\max}$ and iteration tolerance κ.
Output: Output weight $\boldsymbol{W}$.
Initialization: $\boldsymbol{W}_{(0)}$, and iterations $t = 1$.
1. Generate feature layer ($\boldsymbol{F}^P$) by Eq. (1);
2. Generate enhancement layer ($\boldsymbol{E}^Q$) by Eq. (2);
3. Calculate hidden layer $\boldsymbol{A} = \left[\boldsymbol{F}^P, \boldsymbol{E}^Q\right]$;
While $\left\|\boldsymbol{W}_{(t+1)} - \boldsymbol{W}_{(t)}\right\|_2^2 > \kappa$ and $t < t_{\max}$ **do**
4. Update $\bar{\boldsymbol{W}}$ by $\bar{\boldsymbol{W}} = \left[\sqrt{\frac{\sigma_o^2}{\sigma_i^2}} \quad -\boldsymbol{W}^T\right]^T$;
5. Calculate $\boldsymbol{\Lambda}_M$, ϑ and $\boldsymbol{\Lambda}_{MA}$;
6. Calculate s_M and γ_M;
7. Update $\boldsymbol{W}_{(t)}$ by Eq. (20);
8. $t = t + 1$;
end while
9. Set $\boldsymbol{W} = \boldsymbol{W}_{(t)}$.

Algorithm 3 MMTC-BLSb.

Input: Training data $\boldsymbol{X}$ and $\boldsymbol{Y}$, regularization parameters λ, kernel width σ, input and output noise variances σ_i^2 and σ_o^2, maximum iteration number $t_{\max}$ and iteration tolerance κ.
Output: Output weight $\boldsymbol{W}$.
Initialization: $\boldsymbol{W}_{(0)}$, and iterations $t = 1$.
1. Generate feature layer ($\boldsymbol{F}^P$) by Eq. (1);
2. Generate enhancement layer ($\boldsymbol{E}^Q$) by Eq. (2);
3. Calculate hidden layer $\boldsymbol{A} = \left[\boldsymbol{F}^P, \boldsymbol{E}^Q\right]$;
While $\left\|\boldsymbol{W}_{(t+1)} - \boldsymbol{W}_{(t)}\right\|_2^2 > \kappa$ and $t < t_{\max}$ **do**
4. Update $\bar{\boldsymbol{W}}$ by $\bar{\boldsymbol{W}} = \left[\sqrt{\frac{\sigma_o^2}{\sigma_i^2}} \quad -\boldsymbol{W}^T\right]^T$;
5. Calculate $\boldsymbol{\Lambda}_M$ and ϑ;
6. Calculate s_{MB} and γ_{MB};
7. Update $\boldsymbol{W}_{(t)}$ by Eq. (24);
8. $t = t + 1$;
end while
9. Set $\boldsymbol{W} = \boldsymbol{W}_{(t)}$.

one has

$$\begin{aligned} \boldsymbol{W}_{(t)} &= h\left(\boldsymbol{W}_{(t-1)}\right) \\ h(\boldsymbol{W}) &= \left(\boldsymbol{A}^T \boldsymbol{\Lambda}_{MA} \boldsymbol{A} + \gamma_M \boldsymbol{I}\right)^{-1} \boldsymbol{A}^T \boldsymbol{\Lambda}_{MA} \boldsymbol{Y} \end{aligned} \tag{20}$$

Algorithm 2 presents the detailed training procedure of MMTC-BLSa.

In contrast to MMTC-BLSa, MMTC-BLSb does not directly incorporate the M-estimator into the construction of the objective function. Instead, it employs the weighting function of the M-estimator to weight the maximum total correntropy in formulating the objective function, that is,

$$\mathcal{J}_{MB} = \arg\max_{\boldsymbol{W}} \sum_{i=1}^{N} \vartheta_i \exp\left(-\frac{\left\|\boldsymbol{A}_i \boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2 \left\|\bar{\boldsymbol{W}}\right\|_2^2}\right) - \frac{\lambda}{2}\|\boldsymbol{W}\|_2^2 \tag{21}$$

Similar to MTC-BLS, by computing the gradient of $\mathcal{J}_{MB}$ with respect to $\boldsymbol{W}$ and expressing it in matrix form, we can obtain that

$$\frac{\partial \mathcal{J}_{MB}}{\partial \boldsymbol{W}} = -\frac{1}{\sigma^2 \left\|\bar{\boldsymbol{W}}\right\|_2^2}\left(\boldsymbol{A}^T \vartheta \boldsymbol{\Lambda}_M (\boldsymbol{A}\boldsymbol{W} - \boldsymbol{Y}) - s_{MB} \boldsymbol{W}\right) - \lambda \boldsymbol{W} \tag{22}$$

where

$$s_{MB} = \sum_{i=1}^{N} \frac{\vartheta_i \left\|\boldsymbol{A}_i \boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{\left\|\bar{\boldsymbol{W}}\right\|_2^2} \exp\left(-\frac{\left\|\boldsymbol{A}_i \boldsymbol{W} - \boldsymbol{Y}_i\right\|_2^2}{2\sigma^2 \left\|\bar{\boldsymbol{W}}\right\|_2^2}\right)$$

Then, it is easy to know that

$$\boldsymbol{W} = \left(\boldsymbol{A}^T \vartheta \boldsymbol{\Lambda}_M \boldsymbol{A} + \gamma_{MB} \boldsymbol{I}\right)^{-1} \boldsymbol{A}^T \vartheta \boldsymbol{\Lambda}_M \boldsymbol{Y} \tag{23}$$

where $\gamma_{MB} = \lambda\sigma^2 \left\|\bar{\boldsymbol{W}}\right\|_2^2 - s_{MB}$. Likewise, the output weights $\boldsymbol{W}$ are optimized using the fixed-point iteration method, *i.e.*,

$$\begin{aligned} \boldsymbol{W}_{(t)} &= h\left(\boldsymbol{W}_{(t-1)}\right) \\ h(\boldsymbol{W}) &= \left(\boldsymbol{A}^T \vartheta \boldsymbol{\Lambda}_M \boldsymbol{A} + \gamma_{MB} \boldsymbol{I}\right)^{-1} \boldsymbol{A}^T \vartheta \boldsymbol{\Lambda}_M \boldsymbol{Y} \end{aligned} \tag{24}$$

We also give a detailed training procedure of MMTC-BLSb in Algorithm 3.

Both in MMTC-BLSa and MMTC-BLSb, the M-estimator plays a crucial role. Table 1 lists several commonly used M-estimators (Guo & Xu, 2023) along with their cost functions and the corresponding weighting functions. This paper presents the derivation of the weighting function using the Cauchy function as an example and illustrates the role of the adjustment parameter. For the Cauchy function, its cost function is given by

$$\rho(e) = \frac{p^2}{2} \log\left(1 + \left(\frac{e}{p}\right)^2\right) \tag{25}$$

where p is a threshold of the M-estimator, representing its tolerance to training errors. Its value has a significant impact on the robustness and generalization performance of MMTC-BLSa and MMTC-BLSb. Following robust regression theory (Rousseeuw & Leroy, 1987), the threshold p can be calculated as

$$p = \text{Tuning} \cdot \frac{\text{med}(|\boldsymbol{e}|)}{0.6745} \tag{26}$$

where med$(|\cdot|)$ denotes the median operation, and the constant 0.6745 is used to ensure that the threshold p is unbiased under Gaussian error conditions. Following the calculation rule of the weighting function presented above, the weighting function corresponding to the Cauchy function can be computed as

$$\vartheta(e) = \frac{1}{1 + \left(\frac{e}{p}\right)^2} \tag{27}$$

Similarly, the weighting functions corresponding to other M-estimators can also be calculated.

3.3. Robustness analysis of algorithms

Compared with the analytical solution of standard BLS, the robustness of the proposed algorithms stems from the weight matrix introduced during the fixed-point iteration process. In this section, we analyze the robustness mechanisms of MTC-BLS and its M-estimator variants (MMTC-BLSa/b) as follows.

1) **Robustness of MTC-BLS:**

In the MTC-BLS algorithm, the update of the output weights $\boldsymbol{W}$ depends on the MTC weighting matrix $\boldsymbol{\Lambda}_M$, whose diagonal elements actually serve as the robust weights assigned to each sample. Fig. 2 shows the evolutionary relationship between MTC-BLS and other related improved BLS methods. As illustrated in Fig. 2, MTC-BLS is not a simple superposition of methods, but a unified framework that integrates the advantages of total least squares (TLS) and the maximum correntropy criterion (MCC).

- **Handling input noise (TLS mechanism):** Standard BLS and MCC-based variants usually assume that noise exists only in the output $\boldsymbol{Y}$. However, as illustrated in Fig. 2, the transition from standard BLS to TLS-based BLS introduces the denominator term $\|\bar{\boldsymbol{W}}\|_2^2$ (originating from the concept of "weighted residuals"), which allows the model to normalize errors relative to the input weights. This structure enables MTC-BLS to effectively handle noise in the input data matrix, a capability that traditional MCC-based methods lack.

Table 1
Several commonly used M-estimators.

Type	Cost function ρ	Weighting function ϑ	Tuning*
Huber (Li & Swetits, 1998)	$\begin{cases} \frac{e^2}{2}, \left\|\frac{e}{p}\right\| \le 1 \\ p\|e\| - \frac{p^2}{2}, \left\|\frac{e}{p}\right\| > 1 \end{cases}$	$\begin{cases} 1, \left\|\frac{e}{p}\right\| \le 1 \\ \frac{p}{\|e\|}, \left\|\frac{e}{p}\right\| > 1 \end{cases}$	1.345
Cauchy (Zhou et al., 2017)	$\frac{p^2}{2}\log\left(1+\left(\frac{e}{p}\right)^2\right)$	$\frac{1}{1+\left(\frac{e}{p}\right)^2}$	2.385
Bisquare (Fouad et al., 2010)	$\begin{cases} \frac{p^2}{6}\left(1-\left(1-\left(\frac{e}{p}\right)^2\right)^3\right), \left\|\frac{e}{p}\right\| \le 1 \\ \frac{p^2}{6}, \left\|\frac{e}{p}\right\| > 1 \end{cases}$	$\begin{cases} \left(1-\left(\frac{e}{p}\right)^2\right)^2, \left\|\frac{e}{p}\right\| \le 1 \\ 0, \left\|\frac{e}{p}\right\| > 1 \end{cases}$	4.685
Welsch (Lee et al., 2009)	$\frac{p^2}{2}\left(1-\exp\left(-\left(\frac{e}{p}\right)^2\right)\right)$	$\exp\left(-\left(\frac{e}{p}\right)^2\right)$	2.985

* Tuning denotes the adjustment parameter of the M-estimator.

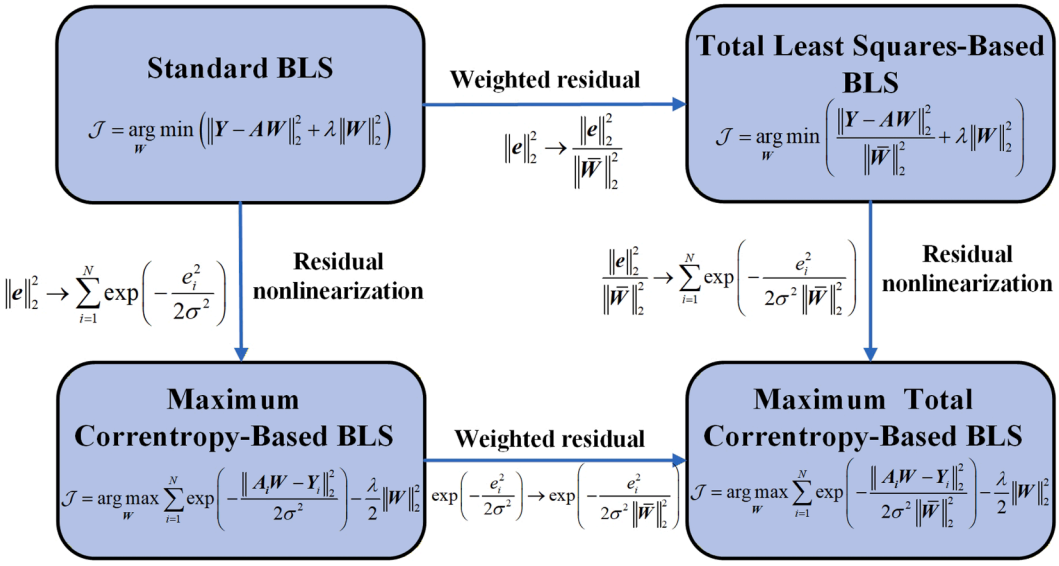


Fig. 2. The relationship between the cost (utility) function of BLS, TLS-BLS, MCC-BLS and MTC-BLS.

- **Outlier suppression (MCC mechanism):** MTC-BLS inherits the "residual nonlinearity" property of MCC (as shown in the lower part of Fig. 2). By mapping the TLS-weighted geometric errors into the kernel space through the exponential function $\exp(\cdot)$, the algorithm effectively truncates large errors caused by outliers. As a result, MTC-BLS achieves dual robustness against both input noise and non-Gaussian outliers.

2) **Robust compensation by the M-estimator (MMTC-BLSa/b):** Although MTC-BLS demonstrates excellent robustness, its performance depends on the proper selection of the kernel width σ. The introduction of the M-estimator in MMTC-BLSa and MMTC-BLSb provides a "double-insurance" mechanism.

- **Independent robustness:** Taking the Huber estimator as an example, the M-estimator directly defines the weighting function $\vartheta(\cdot)$ based on the residual magnitude. When a residual $|e_i|$ exceeds the threshold, the sample is identified as a potential outlier and its weight is reduced. This mechanism is independent of the kernel width σ.
- **Adaptive correction:** During the fixed-point iteration, if the kernel width σ deviates from its optimal value (resulting in insufficient suppression of outliers by the MTC criterion), the resulting increase in training error is captured by the M-estimator. The M-estimator then enforces a reduction of the sample weights, effectively compensating for the performance degradation caused by the deviation of the kernel parameter. This ensures that MMTC-BLSa/b maintains high stability across a broader range of parameter settings.

4. Computational complexity and convergence properties

This section will focus on the discussion of the computational complexity and convergence properties of the proposed algorithms. Note that in this section, N denotes the number of samples available for training, N_w the number of feature node groups, N_f the number of nodes in each feature node group, N_e the total number of enhancement nodes, N_k the total number of nodes in the network with $N_k = N_w \cdot N_f + N_e$, and T_i the actual number of iterations required to complete the fixed-point procedure.

4.1. Computational complexity

From Algorithms 1–3, it is easy to observe that the computational complexity of the proposed MTC-BLS, MMTC-BLSa and MMTC-BLSb mainly consists of three parts, *i.e.*, the generation of feature nodes $\boldsymbol{F}^P$, the generation of enhanced nodes $\boldsymbol{E}^Q$, and the fixed-point iteration of output weights $\boldsymbol{W}$. In the proposed three algorithms, the computational complexity of generating feature nodes can be expressed as $\mathcal{O}(N \cdot N_w \cdot N_f)$, while that of generating enhancement nodes can be expressed as $\mathcal{O}(N_w \cdot N_f \cdot N_e)$. The computational complexity of the fixed-point iteration process for output weights $\boldsymbol{W}$ varies slightly among the three algorithms. Here, following Zheng et al. (2023), the computational complexity of computing each kernel estimate in the maximum total correntropy criterion is taken as $\mathcal{O}(1)$. Then, we have separately analyzed the computational costs of fixed points in each iteration for the three algorithms as follows:

1) **MTC-BLS**: In Step 4, the computational complexity of calculating $\bar{\boldsymbol{W}}$ is $\mathcal{O}(N)$. In Step 5, when calculating $\boldsymbol{\Lambda}_M$, it is necessary to compute the errors $\boldsymbol{e}$ and the kernel function values for all samples, with a computational complexity of $\mathcal{O}(N \cdot N_k)$. In Step 6, since the error $\boldsymbol{e}$ has already been calculated, the costs of calculating s_M and γ_M are $\mathcal{O}(N)$ and $\mathcal{O}(1)$ respectively. In the final step 7, since it involves an inverse operation of a $N_k \times N_k$ matrix, the complexity of solving for the output weights $\boldsymbol{W}$ is $\mathcal{O}(N \cdot N_k^2 + N_k^3)$.

2) **MMTC-BLSa**: In steps 4, 5 and 6, the computational cost required for calculating $\boldsymbol{\Lambda}_M$, s_M and γ_M is the same as that of MTC-BLS. Moreover, the complexities of determining $\boldsymbol{\Lambda}_{MA}$ and $\boldsymbol{\vartheta}$ are $\mathcal{O}(N \cdot N_k)$ and $\mathcal{O}(N)$, respectively. And for solving output weights $\boldsymbol{W}$, the computational complexity is also $\mathcal{O}(N \cdot N_k^2 + N_k^3)$.

3) **MMTC-BLSb**: Indeed, for MMTC-BLSb, its computational complexity differs from MTC-BLS and MMTC-BLSa only in the 7th step. While solving for s_{MB} and γ_{MB}, their computational complexities are $\mathcal{O}(N)$ and $\mathcal{O}(1)$, respectively. Then, the computational cost for calculating the output weights can be expressed as $\mathcal{O}(N \cdot N_k^2 + N_k^3)$.

Suppose that after training for T_i ($T_i \le t_{\max}$) cycles, the model converges to the final solution. By combining the above analysis, it can be observed that the additional costs of MMTC-BLSa and MMTC-BLSb compared to MTC-BLS are $\mathcal{O}(N \cdot N_k)$, which is very small in comparison to $\mathcal{O}(N \cdot N_k^2)$ or $\mathcal{O}(N_k^3)$ and can be disregarded. Therefore, it can be concluded that the computational complexities of the three algorithms proposed in this paper fall within the same order, and can be determined as $\mathcal{O}(T_i \cdot N \cdot N_k^2 + T_i \cdot N_k^3)$. Furthermore, when $N \le N_k$, the computational costs of the algorithms mainly depends on T_i and N_k, which are $\mathcal{O}(T_i \cdot N_k^3)$. Conversely, the computational complexities of the algorithms is simultaneously influenced by T_i, N and N_k, which are equal to $\mathcal{O}(T_i \cdot N \cdot N_k^2)$.

For the three algorithms, in practice, the smaller the iteration cycles T_i is, the lower the computational costs will be. From on our experimental results, we can find that the number of iterations required for the algorithms to reach the convergence solution is usually less than 10 (as shown in Fig. 6), indicates that the increase in computational costs caused by repeated iterations is limited.

Remark 1. It is worth emphasizing that although the fixed-point iteration method adopted in this paper involves an iterative process, it is fundamentally different from the backpropagation (BP) algorithm used in deep neural networks. First, the proposed fixed-point iteration is derived from the zero-gradient condition of the objective function and can be interpreted as a reweighted least-squares solver. Compared with the first-order gradient descent strategy employed by BP, fixed-point iteration generally exhibits much faster convergence. In practice, the proposed method converges to a stable solution within fewer than ten iterations, whereas BP typically requires hundreds of training epochs. Second, the proposed approach does not require tuning hyperparameters such as the learning rate, while in BP-based methods, hyperparameter tuning is often sensitive and time-consuming. Finally, we strictly adhere to BLS architecture by keeping the input weights and enhancement weights randomly generated and fixed, and optimizing only the output weights. This design ensures that the proposed model is significantly more lightweight compared to approaches that rely on full-network fine-tuning.

4.2. Convergence properties

The previous subsection discussed the computational complexity of the three proposed algorithms. In this subsection, we will conduct a detailed analysis of the convergence characteristics of the three proposed improved algorithms. Since MTC-BLS, MMTC-BLSa and MMTC-BLSb all adopt the fixed-point iteration method for training, their convergence mainly depends on the fixed-point iteration method. Banach's fixed-point theorem (Xie et al., 2020) (*i.e.*, Theorem 1) is introduced to analyze the convergence of the three proposed algorithms.

Theorem 1. *The convergence of the fixed-point iteration method for MTC-BLS can be effectively guaranteed if there exists a real number $\xi > 0$ and another real number $\varrho \in (0,1)$ such that the initial weight $\boldsymbol{W}_{(0)}$ satisfies $\|\boldsymbol{W}_{(0)}\|_1 \le \xi$, and the following condition holds for any $\boldsymbol{W} \in \{\boldsymbol{W} \mid \|\boldsymbol{W}\| \le \xi, \boldsymbol{W} \in \mathbb{R}^N\}$,*

$$\begin{cases} \|h(\boldsymbol{W})\|_1 \le \xi \\ \left\|\nabla_{\boldsymbol{W}} h(\boldsymbol{W})\right\|_1 \le \varrho \end{cases} \tag{28}$$

where $\|\cdot\|_1$ represents the l_1-norm of a vector or a matrix, and the symbol $\nabla_{\boldsymbol{W}} h(\boldsymbol{W})$ is defined as the Jacobian matrix of $h(\boldsymbol{W})$ with respect to $\boldsymbol{W}$, that is,

$$\nabla_{\boldsymbol{W}} h(\boldsymbol{W}) = \left[\frac{\partial h(\boldsymbol{W})}{\partial \boldsymbol{W}_1} \quad \frac{\partial h(\boldsymbol{W})}{\partial \boldsymbol{W}_2} \quad \cdots \quad \frac{\partial h(\boldsymbol{W})}{\partial \boldsymbol{W}_N}\right] \tag{29}$$

Here,

$$\begin{aligned} \frac{\partial h(\boldsymbol{W})}{\partial \boldsymbol{W}_i} &= \frac{\partial}{\partial \boldsymbol{W}_i}\left[\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y}\right] \\ &= -\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\frac{\partial\boldsymbol{\Lambda}_M}{\partial\boldsymbol{W}_i}\boldsymbol{A}h(\boldsymbol{W}) + \left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\frac{\partial\boldsymbol{\Lambda}_M}{\partial\boldsymbol{W}_i}\boldsymbol{Y} \\ &= -\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{A}_i\right)h(\boldsymbol{W}) \\ &\quad + \left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{Y}_i \end{aligned} \tag{30}$$

Note that in Eq. 30, since s_M is always close to 0, γ_M can be regarded as bounded. According to reference Zhao et al. (2025), γ_M is treated as a constant during the processing. To provide sufficient conditions for MTC-BLS based on Banach's fixed-point theorem, the following Theorem 2 is proposed.

Theorem 2. *If $\sigma \ge \max\{\sigma^\star, \sigma^*\}$ and $\sigma \in (0, +\infty)$, then for any*

$$\boldsymbol{W} \in \{\boldsymbol{W} \mid \|\boldsymbol{W}\| \le \xi, \boldsymbol{W} \in \mathbb{R}^N\}$$

the following conditions are satisfied:

$$0 < \|h(\boldsymbol{W})\|_1 \le \xi \quad \text{and} \quad \|\nabla_{\boldsymbol{W}} h(\boldsymbol{W})\|_1 \le \varrho$$

where $\sigma^\star$ and σ^* denote the solutions to the equations $\Phi(\sigma^\star) = \xi$ and $\Psi(\sigma^*) = \varrho$, respectively. $\Phi(\sigma^\star)$ and $\Psi(\sigma^*)$ are given by Eqs. (31) and (32), where $\zeta_{\min}(\cdot)$ denotes the minimum eigenvalue of the matrix, and $\tau_i = \frac{(\xi\|\boldsymbol{A}_i\|_1 + |\boldsymbol{Y}_i|)^2}{\|\bar{\boldsymbol{W}}\|_2^2}$.

$$\Phi(\sigma) = \frac{\sqrt{N}\sum_{i=1}^{N}\left\|\boldsymbol{A}_i^T\right\|_1|\boldsymbol{Y}_i|}{\zeta_{\min}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\exp\left(-\frac{\tau_i}{2\sigma^2}\right)\boldsymbol{A}_i\right) + \lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2 - \sum_{i=1}^{N}\tau_i\exp\left(-\frac{\tau_i}{2\sigma^2}\right)} \tag{31}$$

$$\Psi(\sigma) = \frac{\frac{\sqrt{N}}{\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\sum_{i=1}^{N}\left(\xi\left\|\boldsymbol{A}_i^T\boldsymbol{A}_i\right\|_1 + \left\|\boldsymbol{A}_i^T\right\|_1|\boldsymbol{Y}_i|\right)\left(|\boldsymbol{A}_{ii}|(\xi\|\boldsymbol{A}_i\| + |\boldsymbol{Y}_i|) + \frac{|\bar{\boldsymbol{W}}|}{\|\bar{\boldsymbol{W}}\|_2^2}(\xi\|\boldsymbol{A}_i\|_1 + |\boldsymbol{Y}_i|)^2\right)}{\zeta_{\min}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\exp\left(-\frac{\tau_i}{2\sigma^2}\right)\boldsymbol{A}_i\right) + \lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2 - \sum_{i=1}^{N}\tau_i\exp\left(-\frac{\tau_i}{2\sigma^2}\right)} \tag{32}$$

Proof. From the compatibility of the induced norm of the matrix and the corresponding vector norm, one has

$$\begin{aligned} \|h(\boldsymbol{W})\|_1 &= \left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y}\right\|_1 \\ &\le \left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\right\|_1\left\|\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y}\right\|_1 \end{aligned} \tag{33}$$

For $\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\right\|_1$, based on $|e_i|=|\boldsymbol{A}_i\boldsymbol{W}-\boldsymbol{Y}_i|\le\|\boldsymbol{A}_i\|_1\|\boldsymbol{W}\|_1+|\boldsymbol{Y}_i|\le\xi\|\boldsymbol{A}_i\|_1+|\boldsymbol{Y}_i|$ and the properties of the matrix, it is not difficult to deduce that

$$\begin{aligned}&\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\right\|_1\\&\le\sqrt{N}\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\right\|_2\\&\le\frac{\sqrt{N}}{\zeta_{\min}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\exp\left(-\frac{\tau_i}{2\sigma^2}\right)\boldsymbol{A}_i\right)+\lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2-\sum_{i=1}^{N}\tau_i\exp\left(-\frac{\tau_i}{2\sigma^2}\right)}\end{aligned}\tag{34}$$

For $\|\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y}\|_1$, since the l_1-norm of a vector is convex and $\sum_{i=1}^{N}\exp\left(-\frac{\|\boldsymbol{A}_i\boldsymbol{W}-\boldsymbol{Y}_i\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right)\le 1$, it follows that

$$\left\|\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{Y}\right\|_1\le\sum_{i=1}^{N}\left\|\boldsymbol{A}_i^T\boldsymbol{\Lambda}_{Mii}\boldsymbol{Y}_i\right\|_1\le\sum_{i=1}^{N}\left\|\boldsymbol{A}_i^T\right\|_1|\boldsymbol{Y}_i|\tag{35}$$

From Eqs. (33) to (35), it can be seen that $\|h(\boldsymbol{W})\|_1$ satisfies Eq. (36). For $\sigma\in(0,+\infty)$, $\Phi(\sigma)$ is a monotonically decreasing function, which satisfies $\lim_{x\to0^+}\Phi(\sigma)=\infty$ and $\lim_{x\to\infty}\Phi(\sigma)=0$. Thus, $\Phi(\sigma^\star)=\xi$ has a unique solution $\sigma^\star$, and when $\sigma>\sigma^\star$, it holds that $\|h(\boldsymbol{W})\|_1\le\Phi(\sigma)\le\xi$.

$$\begin{aligned}\|h(\boldsymbol{W})\|_1&\le\frac{\sqrt{N}\sum_{i=1}^{N}\left\|\boldsymbol{A}_i^T\right\|_1|\boldsymbol{Y}_i|}{\zeta_{\min}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\exp\left(-\frac{\tau_i}{2\sigma^2}\right)\boldsymbol{A}_i\right)+\lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2-\sum_{i=1}^{N}\tau_i\exp\left(-\frac{\tau_i}{2\sigma^2}\right)}\\&=\Phi(\sigma)\end{aligned}\tag{36}$$

For the second condition in Theorem 1, if for all i, it holds that $\left\|\frac{\partial h(\boldsymbol{W})}{\partial\boldsymbol{W}_i}\right\|_1\le\varrho$, then it must be that $\|\nabla_{\boldsymbol{W}}h(\boldsymbol{W})\|_1\le\varrho$. According to Eq. (30), we have

$$\begin{aligned}&\left\|\frac{\partial h(\boldsymbol{W})}{\partial\boldsymbol{W}_i}\right\|_1\\&\le\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{A}_i\right)h(\boldsymbol{W})\right\|_1\\&+\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{Y}_i\right\|_1\end{aligned}\tag{37}$$

By the convexity of the vector l_1-norm and $\|h(\boldsymbol{W})\|_1\le\xi$, the derivation of the first term on the right side of the inequality in Eq. (37) can be carried out as follows

$$\begin{aligned}&\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{A}_i\right)h(\boldsymbol{W})\right\|_1\\&\le\xi\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\right\|_1\sum_{i=1}^{N}\left\|\boldsymbol{A}_i^T\boldsymbol{A}_i\right\|_1\left|\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\right|\end{aligned}\tag{38}$$

Since $|e_i|=|\boldsymbol{A}_i\boldsymbol{W}-\boldsymbol{Y}_i|\le\|\boldsymbol{A}_i\|_1\|\boldsymbol{W}\|_1+|\boldsymbol{Y}_i|\le\xi\|\boldsymbol{A}_i\|_1+|\boldsymbol{Y}_i|$ and $\sum_{i=1}^{N}\exp\left(-\frac{\|\boldsymbol{A}_i\boldsymbol{W}-\boldsymbol{Y}_i\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right)\le 1$, we can obtain that

$$\begin{aligned}\left|\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\right|&=\left|\frac{\partial}{\partial\boldsymbol{W}_i}\left(\exp\left(-\frac{\|\boldsymbol{A}_i\boldsymbol{W}-\boldsymbol{Y}_i\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right)\right)\right|\\&\le\frac{1}{\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}|\boldsymbol{A}_{ii}|\left(\xi\|\boldsymbol{A}_i\|+|\boldsymbol{Y}_i|\right)+\frac{1}{\sigma^2\|\bar{\boldsymbol{W}}\|_2^4}|\bar{\boldsymbol{W}}|\left(\xi\|\boldsymbol{A}_i\|_1+|\boldsymbol{Y}_i|\right)^2\end{aligned}\tag{39}$$

Similarly, for the second term on the right side in Eq. (37), there is

$$\begin{aligned}&\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{Y}_i\right\|_1\\&\le\left\|\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M\boldsymbol{A}+\gamma_M\boldsymbol{I}\right)^{-1}\right\|_1\sum_{i=1}^{N}\left\|\boldsymbol{A}_i^T\right\|_1\left|\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\right||\boldsymbol{Y}_i|\end{aligned}\tag{40}$$

Then, by summarizing Eqs. (34), and (37) to (40), it can be easily concluded Eq. (41). Obviously, for $\sigma\in(0,+\infty)$, $\Psi(\sigma)$ is a monotonically decreasing function, which satisfies $\lim_{x\to0^+}\Psi(\sigma)=\infty$ and $\lim_{x\to\infty}\Psi(\sigma)=0$. Thus, $\Psi(\sigma^*)=\xi$ has a unique solution σ^*, and when $\sigma>\sigma^*$, it holds that $\left\|\frac{\partial h(\boldsymbol{W})}{\partial\boldsymbol{W}_i}\right\|_1\le\Psi(\sigma)\le\varrho<1$, then we have $\|\nabla_{\boldsymbol{W}}h(\boldsymbol{W})\|_1\le\Psi(\sigma)\le\varrho<1$.

$$\begin{aligned}&\left\|\frac{\partial h(\boldsymbol{W})}{\partial\boldsymbol{W}_i}\right\|_1\\&\le\frac{\sqrt{N}\left(\xi\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{A}_i\right)+\sum_{i=1}^{N}\boldsymbol{A}_i^T\frac{\partial\boldsymbol{\Lambda}_{Mii}}{\partial\boldsymbol{W}_i}\boldsymbol{Y}_i\right)}{\zeta_{\min}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\exp\left(-\frac{\tau_i}{2\sigma^2}\right)\boldsymbol{A}_i\right)+\lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2-\sum_{i=1}^{N}\tau_i\exp\left(-\frac{\tau_i}{2\sigma^2}\right)}\\&=\frac{\frac{\sqrt{N}}{\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\sum_{i=1}^{N}\left(\xi\|\boldsymbol{A}_i^T\boldsymbol{A}_i\|_1+\|\boldsymbol{A}_i^T\|_1|\boldsymbol{Y}_i|\right)\left(|\boldsymbol{A}_{ii}|\left(\xi\|\boldsymbol{A}_i\|+|\boldsymbol{Y}_i|\right)+\frac{|\bar{\boldsymbol{W}}|}{\|\bar{\boldsymbol{W}}\|_2^2}\left(\xi\|\boldsymbol{A}_i\|_1+|\boldsymbol{Y}_i|\right)^2\right)}{\zeta_{\min}\left(\sum_{i=1}^{N}\boldsymbol{A}_i^T\exp\left(-\frac{\tau_i}{2\sigma^2}\right)\boldsymbol{A}_i\right)+\lambda\sigma^2\|\bar{\boldsymbol{W}}\|_2^2-\sum_{i=1}^{N}\tau_i\exp\left(-\frac{\tau_i}{2\sigma^2}\right)}\\&=\Psi(\sigma)\end{aligned}\tag{41}$$

Based on the above discussion, from Theorems 1 and 2, it can be concluded that the fixed-point MTC-BLS algorithm converges within the range of $\boldsymbol{W}\in\{\boldsymbol{W}\mid\|\boldsymbol{W}\|\le\xi,\boldsymbol{W}\in\mathbb{R}^N\}$, while $\sigma\ge\max\{\sigma^\star,\sigma^*\}$. It should be noted that the convergence conditions provided in this paper are sufficient but not necessary. In fact, the algorithm may still converge in practice even when the requirement $\sigma\ge\max\{\sigma^\star,\sigma^*\}$ is not satisfied (Zhao & Lu, 2024). Moreover, the convergence analyses of the two proposed variants, MMTC-BLSa and MMTC-BLSb, follow essentially the same procedure as that of MTC-BLS. Considering the conciseness of the article, we will not repeat it here. Interested readers conduct a theoretical analysis of their convergence based on the same analytical approach as the MTC-BLS. □

Remark 2. It should be noted that the sufficient conditions derived in Theorems 1 and 2 involve computing the spectral norm and eigenvalues of data-related matrices, which can be computationally expensive for large-scale datasets. These theorems primarily provide a theoretical guarantee for the existence of a convergence range for the kernel width σ. They reveal a mechanism whereby sufficiently large σ facilitates the achievement of a contraction mapping. In practical applications, these conditions serve as qualitative guidance, while the precise selection of σ is typically determined efficiently through cross-validation or heuristic search within a reasonable range to balance convergence stability and predictive accuracy.

5. Experimental results and analysis

In this section, experiments on seven regression datasets, two time series datasets and four image datasets are conducted to evaluate the performance of the three proposed types of novel BLS. The information of the regression datasets and time series datasets used is shown in Table 2. Note that to ensure the unbiasedness of the experimental results and eliminate the influence of different variable scales, all data are normalized to the range of $(0,1)$ before the experiments. And all experimental results are obtained on a platform equipped with an NVIDIA GeForce RTX 4090 GPU and 256 GB of RAM, with the code implemented in Python 3.10 under the CUDA 12.4 environment. To fairly compare the performance of different algorithms, Accuracy is adopted as the evaluation metric for the image datasets, meanwhile, the root mean square error (RMSE) (Yin et al., 2025) and the Robustness Index (RI) (Wang et al., 2026) are adopted as evaluation metrics for the regression datasets and time series datasets, which are calculated as follows, respectively:

$$RMSE=\sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y_{\text{pre}}(i)-y_{\text{true}}(i)\right)^2}\tag{42}$$

Table 2
Datasets specification information description.

Datasets	n_{total}	n_{train}	n_{test}	features
Abalone ()Kelly, Longjohn)	4177	2089	2088	8
Basketball ()Kelly, Longjohn)	96	48	48	4
Mortgage ()Kelly, Longjohn)	1049	525	524	15
Photovoltaic Power1 ()Kelly, Longjohn)	45,000	22,500	22,500	7
Photovoltaic Power2 ()Kelly, Longjohn)	150,000	75,000	75,000	7
SPARK-Raw (Sievers et al., 2026)	80,000	40,000	40,000	7
Batch ()Kelly, Longjohn)	2299	1150	1149	128
Mackey-Glass (Chen et al., 2018)	1994	997	997	7
Sunspots (Clette & Lefèvre, 2015)	3308	1654	1654	4

$$RI_x = \frac{RMSE_0}{RMSE_x} \times 100\% \tag{43}$$

where y_{pre} and y_{true} represent the predicted values and the true values of the target output, respectively. $RMSE_0$ denotes the RMSE under noise-free conditions, and $RMSE_x$ denotes the RMSE at a contamination rate of $x\%$. RMSE represents the accuracy of the model's predictions, with lower values indicating better predictive performance. RI measures the robustness of the model, with higher values indicating better robustness at the same RMSE level.

In this paper, BLS (Chen & Liu, 2018), C-BLS (Zheng et al., 2021), MGC-BLS-Huber (Zhao et al., 2025), GCN-LSTM (Guo, 2026), and UQAE (Zhang et al., 2025) are adopted as baseline methods to verify the superiority of the proposed algorithms. Since different parameter settings have an important impact on BLS, the grid search method is used to select the number of feature node groups N_w, the number of feature nodes for each feature node group N_f, and the number of feature nodes N_e. In all experiments, the optimal combination (N_w, N_f, N_e) of parameters is determined by step size 1, 1, 2 search in the range of $[1, 20] \times [1, 20] \times [1, 200]$. Note that for all the improved BLS methods, the regularization parameter is set to 2^{-30}. For MGC-BLS-Huber, its shape parameter α is determined within the candidate set $\{1, 2, 3, 4\}$ (Zhao et al., 2025). For C-BLS, MGC-BLS-Huber, and the methods proposed in this paper, the kernel width σ is set to 0.1. And for MTC-BLS, MMTC-BLSa, and MMTC-BLSb, the variances of the input noise and output noise in the training samples are set to $\sigma_i^2 = \sigma_o^2 = 0.1$. Furthermore, according to Theorems 1 and 2 in Section 4.2, when the kernel width σ satisfies $\sigma \geq \max\{\sigma^\star, \sigma^*\}$, the three proposed algorithms are guaranteed to converge. Therefore, in this study, σ is set to 0.2 to ensure algorithmic convergence while remaining as close as possible to the optimal value. After obtaining the optimal parameters through grid search, each algorithm is independently executed ten times. The training time and RMSE of each run are recorded, and the mean and standard deviation (STD) of RMSE, along with the average training time, are calculated and presented as the results to compare the performance of different algorithms.

5.1. Experiments on time series datasets

In this subsection, two time series datasets are constructed using the widely used Mackey-Glass chaotic time series system (Fu et al., 2024; Gunasekaran et al., 2025) and the monthly average total Sunspot number prediction data (Clette & Lefèvre, 2015), with their information presented in Table 2. Followingly, we provide a detailed description of the construction of these two time series datasets.

For the Mackey-Glass time series, which exhibits chaotic dynamics, it can be represented by the following differential equation

$$\frac{dx(t)}{dt} = -bx(t) + \frac{ax(t-\tau)}{1 + x(t-\tau)^{10}} \tag{44}$$

where $a=0.1$, $b=0.2$ and $\tau=30$ are set in this study. Following the recommendation of Chen et al. (2018), the dataset is constructed by using the previous seven data points to predict the current point, that is, at time t, the input and output are formed as $x_t = [x(t-7), x(t-6), \ldots, x(t-1)]$ and $y_t = x(t)$, respectively. In our experiments, 2000 samples are drawn according to Eq. (10), and the dataset is split into training and testing sets at a 1:1 ratio.

For the monthly average total Sunspot number prediction data, the dataset is constructed using the monthly mean sunspot numbers from January 1749 to December 2024, with the embedding dimension set to 4 and the delay set to 1. Consistent with the Mackey-Glass dataset, the data is split into training and testing sets at a 1:1 ratio.

5.1.1. Comparative experiments with different types of noise

To comprehensively evaluate the performance of the proposed algorithms in the presence of noise in the input data, different types of noise are simultaneously added to both the inputs and outputs of the training samples to simulate various noisy environments. Specifically, α-stable noise and Gaussian noise are introduced. The α-stable noise sequence is generated from an α-stable distribution $f(t) = e^{-\rho|x|^\mu}$ with parameters $\rho = 0.5$ and $\mu = 1$, while the Gaussian noise sequence is generated from a Gaussian distribution with zero mean and a variance of 0.01. The experimental results of each algorithm on the different time series datasets are presented in Figs. 3 and 4. Here, MMTC-BLSa-Huber denotes the MMTC-BLSa algorithm that employs the Huber function as the M-estimator, and the others follow the same naming convention, which will not be repeated in the subsequent sections. Then the following conclusions can be drawn.

(1) On the noise-free time series data, our proposed methods achieved slightly superior performance. However, when Gaussian noise or α-stable noise was simultaneously added to both the input and output of the training data, the performance of BLS and C-BLS degraded noticeably, whereas MTC-BLS, even without the incorporation of a robust M-estimator, achieved predictive performance comparable to that of MGC-BLS-Huber.

(2) After introducing the M-estimator, both MMTC-BLSa and MMTC-BLSb consistently achieved superior performance compared with the benchmark methods. In addition, the prediction results on the Sunspots dataset further demonstrate the effectiveness of the proposed methods in real-world time series forecasting tasks.

The above results indicate that incorporating the MTC criterion and the robust M-estimator can effectively enhance the ability of BLS to handle noise in both inputs and outputs, thereby improving the overall robustness of the model.

5.1.2. Comparative experiments with different noise contamination levels

To comprehensively analyze the robustness of the three types of improved BLS methods proposed in this paper for time series prediction tasks, we follow the noise and outlier injection strategy described in Chu et al. (2020) to contaminate the input and output of the training samples in the two time series datasets at different levels (*i.e.*, $p = 10\%$, 20%, 30%, 40%, 50% and 60%), while keeping the test data clean. Comparative experiments are then conducted based on these settings. The experimental results are presented in Table 3, and the RI values of each algorithm are further calculated according to these results, as shown in Table 4. From Tables 3 and 4, we can draw the following interesting conclusions.

(1) As the contamination rate increases, RMSE rises for both MTC-BLS and the benchmark methods, while MTC-BLS exhibits a smaller increase, demonstrating superior performance. This indicates that, compared with the methods improved by adopting the MCC criterion, introducing the MTC criterion enhances the ability of BLS to handle noise in the input data of training samples.

(2) After introducing the M-estimator, the predictive performance of MTC-BLSa and MTC-BLSb is significantly improved, indicating that the incorporation of the M-estimator provides a substantial enhancement to the algorithm.

(3) Compared with MMTC-BLSa, MMTC-BLSb achieves superior performance, particularly for the MMTC-BLSb using the Welsch estimator,

Table 3
The experimental results (mean± STD) on time series datasets with different contamination rates (RMSE $\times 10^{-2}$).

Datasets	Noise rate	Methods														
		BLS	C-BLS	MGC-BLS -Huber	GCN-LSTM	UQAE	MTC-BLS	MMTC-BLSa -Huber	MMTC-BLSa -Cauchy	MMTC-BLSa -Bisquare	MMTC-BLSa -Welsch	MMTC-BLSb -Huber	MMTC-BLSb -Cauchy	MMTC-BLSb -Bisquare	MMTC-BLSb -Welsch
Mackey -Glass	0%	20.75±0.05	20.53±0.04	19.33±0.04	19.62±0.14	19.59±0.12	19.57±0.05	18.96±0.03	18.71±0.04	19.07±0.06	18.65±0.02	17.32±0.02	18.36±0.04	19.02±0.12	**16.09±0.06**
	10%	21.03±0.16	20.78±0.13	19.74±0.08	19.23±0.18	19.14±0.14	18.96±0.12	19.01±0.06	19.19±0.13	18.93±0.11	18.73±0.07	17.22±0.14	18.28±0.17	18.94±0.16	**15.92±0.09**
	20%	21.54±0.23	21.04±0.17	19.52±0.19	19.02±0.22	18.97±0.18	18.83±0.11	19.36±0.14	18.57±0.06	**18.41±0.07**	18.67±0.22	17.18±0.13	18.12±0.15	18.78±0.14	18.47±0.08
	30%	21.38±0.37	20.93±0.28	19.57±0.13	19.28±0.26	19.13±0.21	18.99±0.27	19.00±0.14	18.97±0.23	18.69±0.25	18.79±0.12	**15.68±0.17**	18.03±0.21	18.52±0.31	15.72±0.18
	40%	21.93±0.42	21.33±0.36	19.64±0.27	19.52±0.32	19.44±0.27	19.27±0.17	18.81±0.33	18.92±0.14	18.51±0.10	18.84±0.38	17.07±0.16	17.82±0.22	18.37±0.25	**15.58±0.24**
	50%	22.54±0.33	21.84±0.48	19.56±0.19	19.63±0.41	19.78±0.38	19.34±0.41	19.38±0.17	19.17±0.33	18.12±0.12	18.88±0.15	17.03±0.17	17.78±0.24	18.13±0.28	**15.43±0.28**
	60%	22.44±0.53	21.78±0.43	19.61±0.24	19.41±0.36	19.93±0.42	19.33±0.21	19.04±0.48	19.13±0.17	19.43±0.14	18.93±0.51	16.92±0.21	17.68±0.26	18.03±0.36	**15.28±0.33**
Sunspots	0%	19.86±0.23	19.25±0.22	19.27±0.18	19.28±0.11	19.24±0.13	19.22±0.26	18.81±0.18	18.92±0.18	18.86±0.16	18.32±0.19	16.59±0.20	17.43±0.47	18.22±0.13	**16.47±0.18**
	10%	20.03±0.32	19.43±0.28	18.82±0.17	18.88±0.31	18.87±0.28	18.79±0.38	18.81±0.17	18.81±0.31	18.81±0.23	18.44±0.24	16.53±0.25	17.32±0.38	18.18±0.18	**16.38±0.23**
	20%	20.28±0.48	19.64±0.41	18.79±0.19	18.72±0.37	18.63±0.32	18.51±0.25	18.55±0.33	18.61±0.22	18.28±0.19	18.39±0.38	16.48±0.23	17.23±0.33	18.04±0.21	**16.23±0.22**
	30%	20.54±0.43	19.88±0.33	18.76±0.22	18.78±0.34	18.71±0.36	18.53±0.28	18.47±0.24	18.46±0.42	18.37±0.21	18.34±0.28	16.43±0.28	17.18±0.48	17.98±0.23	**16.13±0.38**
	40%	20.73±0.58	20.03±0.48	18.74±0.24	18.89±0.41	18.82±0.42	18.47±0.31	18.47±0.26	18.46±0.26	18.37±0.33	**15.94±0.51**	16.38±0.31	17.08±0.40	17.93±0.28	16.03±0.25
	50%	20.98±0.53	20.23±0.43	18.77±0.41	18.61±0.39	18.98±0.44	18.43±0.33	18.44±0.23	18.39±0.28	18.44±0.25	18.23±0.33	16.33±0.33	**15.83±0.43**	17.83±0.43	15.88±0.27
	60%	21.13±0.63	20.48±0.58	**15.63±0.28**	19.02±0.48	18.83±0.51	18.65±0.43	18.63±0.28	18.64±0.30	18.52±0.27	18.18±0.55	16.23±0.36	16.93±0.46	17.73±0.36	15.73±0.48

Table 4
The robustness index (RI) on time series datasets with different contamination rates (%).

Datasets	Noise rate	Methods													
		BLS	C-BLS	MGC-BLS -Huber	GCN-LSTM	UQAE	MTC-BLS	MMTC-BLSa -Huber	MMTC-BLSa -Cauchy	MMTC-BLSa -Bisquare	MMTC-BLSa -Welsch	MMTC-BLSb -Huber	MMTC-BLSb -Cauchy	MMTC-BLSb -Bisquare	MMTC-BLSb -Welsch
Mackey -Glass	10%	98.67	98.80	97.92	102.03	102.35	**103.22**	99.74	97.50	100.74	99.57	100.58	100.44	100.42	101.07
	20%	96.33	97.58	99.03	103.15	103.27	**103.93**	97.93	100.75	103.59	99.89	100.81	101.32	101.28	87.11
	30%	97.05	98.09	98.77	101.76	102.40	**103.05**	99.79	98.63	102.03	99.25	110.46	101.83	102.70	102.35
	40%	94.62	96.25	98.42	100.51	100.77	101.56	100.80	98.89	103.03	98.99	101.46	103.03	**103.54**	103.27
	50%	92.06	94.00	98.82	99.95	99.04	101.19	97.83	97.60	105.24	98.78	101.70	103.26	**104.91**	104.28
	60%	92.47	94.26	98.57	101.08	98.29	101.24	99.58	97.80	98.15	98.52	102.36	103.85	**105.49**	105.30
Sunspots	10%	99.15	99.07	**102.39**	102.12	101.96	102.29	100.00	100.58	100.27	99.35	100.36	100.64	100.22	100.55
	20%	97.93	98.01	102.55	102.99	103.27	**103.84**	101.40	101.67	103.17	99.62	100.67	101.16	101.00	101.48
	30%	96.69	96.83	102.72	102.66	102.83	**103.72**	101.84	102.49	102.67	99.89	100.97	101.46	101.33	102.11
	40%	95.80	96.11	102.83	102.06	102.23	**104.06**	101.84	102.49	102.67	114.93	101.28	102.05	101.62	102.74
	50%	94.66	95.16	102.66	103.60	101.37	104.29	102.01	102.88	102.28	100.49	101.59	**110.11**	102.19	103.72
	60%	93.99	93.99	**123.29**	101.37	102.18	103.06	100.97	101.50	101.84	100.77	102.22	102.95	102.76	104.70

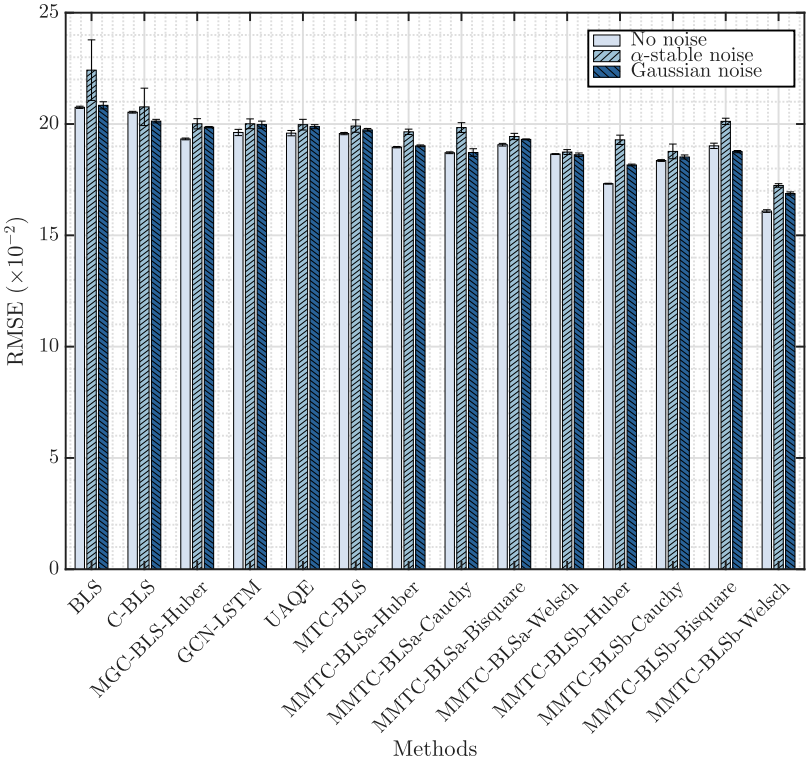


Fig. 3. Experimental results of the Mackey-Glass dataset with different noise types.

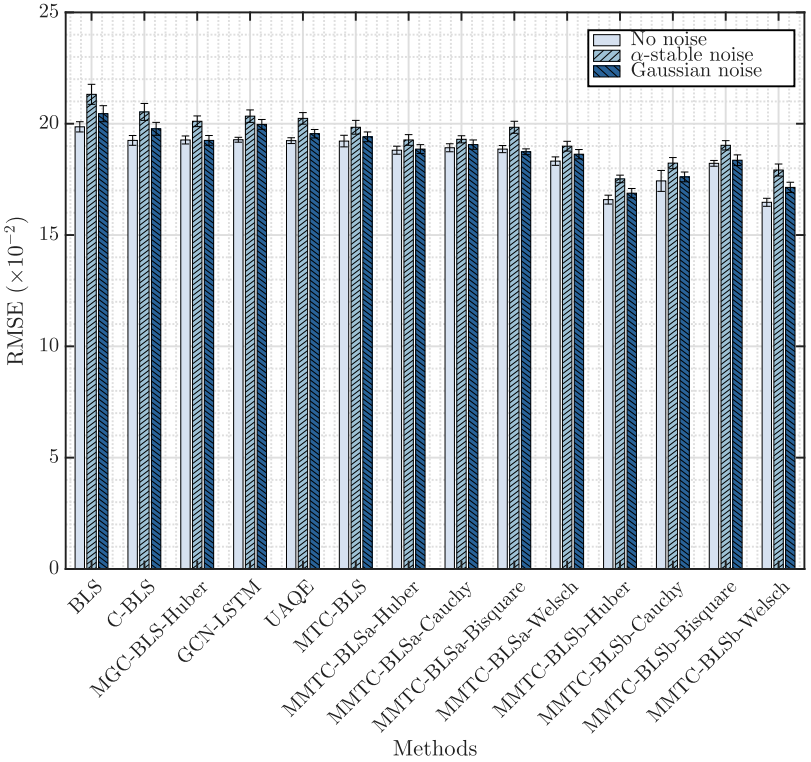


Fig. 4. Experimental results of the Sunspots dataset with different noise types.

which indicates that employing the M-estimator as a weighting factor of the MTC criterion can enhance the model robustness more effectively.

(4) Further observation of the robustness indicator (RI) in Table 4 shows that the proposed algorithms generally achieve higher RI values while maintaining low RMSE. This indicates that, under the same noise level, the proposed methods exhibit smaller performance degradation relative to their noise-free baselines, thereby validating their stability in complex noisy environments.

5.2. Experiments on the regression datasets

The previous subsection discussed the performance of the proposed algorithms on time series data. In this subsection, several real-world datasets from different domains such as marine biology, sports, finance, and photovoltaic power generation are selected to evaluate the robustness of the proposed algorithms in regression tasks.

5.2.1. Comparative experiments with different types of noise

To comprehensively evaluate the regression prediction capabilities of the three types of robust BLS proposed in this paper when both the input and output of the processed data are contaminated with noise, similar to time series data, comparative experiments were conducted by simultaneously adding different noises to the input and output of the training samples. The experimental results of different variants are presented in Fig. 5. Based on these experimental results, we can draw the following conclusions.

(1) The standard BLS is highly sensitive to noise, while its robust variants, C-BLS and MGC-BLS-Huber, exhibit improved performance. The MTC-BLS consistently achieves better prediction accuracy than both C-BLS and MGC-BLS-Huber. This indicates that incorporating the MTC criterion can enhance the algorithm's ability to handle noise in the input data.

(2) Due to the varying characteristics of different M-estimator functions, the performance of MMTC-BLSa and MMTC-BLSb may exhibit minor fluctuations. Nevertheless, it is clear that both MMTC-BLSa and MMTC-BLSb can effectively enhance the predictive capability of MTC-BLS under any circumstances. This indicates that the two M-estimator-based construction strategies we proposed can both effectively improve the algorithm's performance.

(3) MMTC-BLSb achieves better performance than MMTC-BLSa on the Abalone, Basketball, and Photovoltaic Power datasets, indicating that the second M-estimator-based construction strategy proposed in this paper provides greater gains to MTC-BLS, which is consistent with the conclusions drawn earlier.

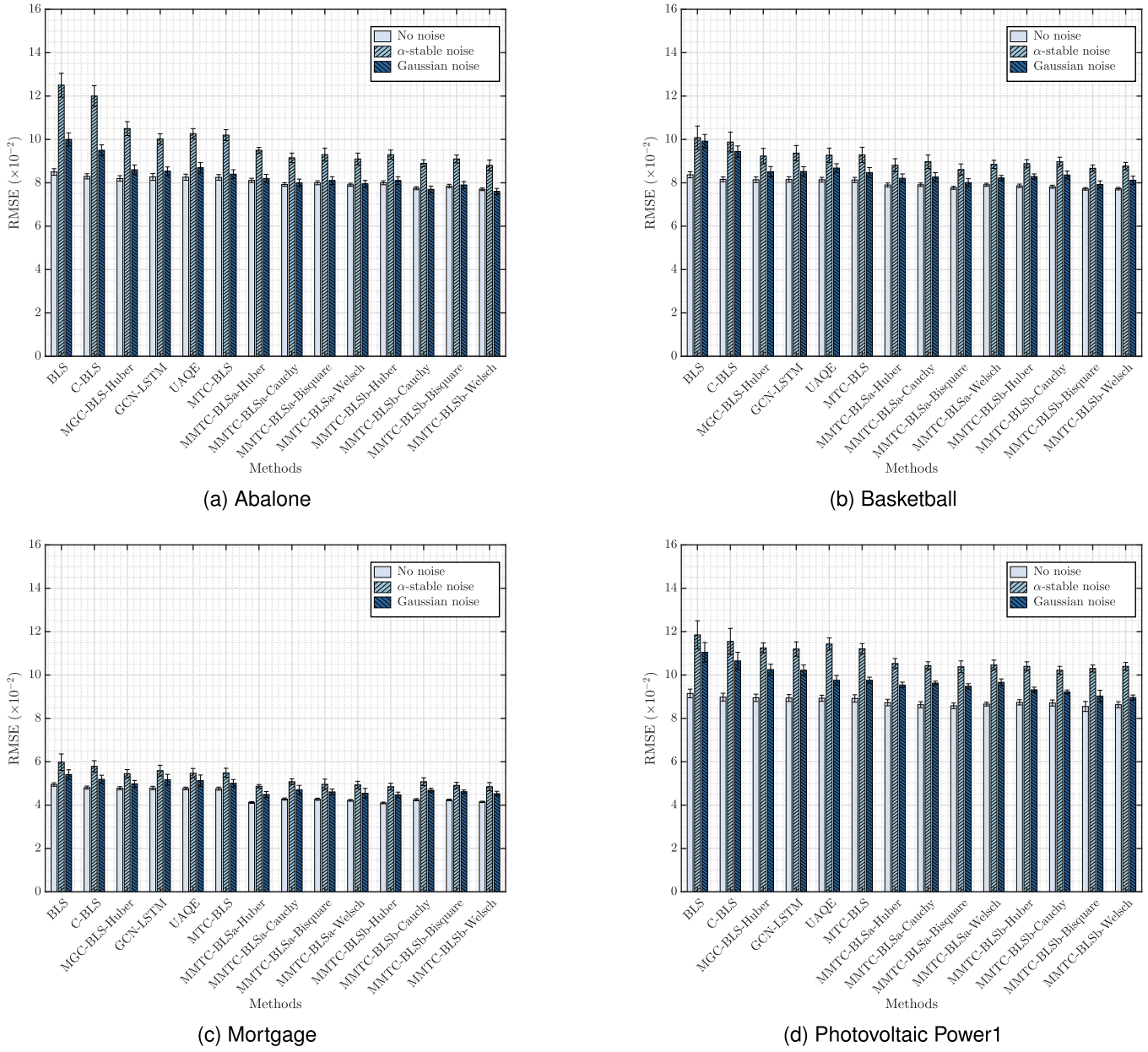
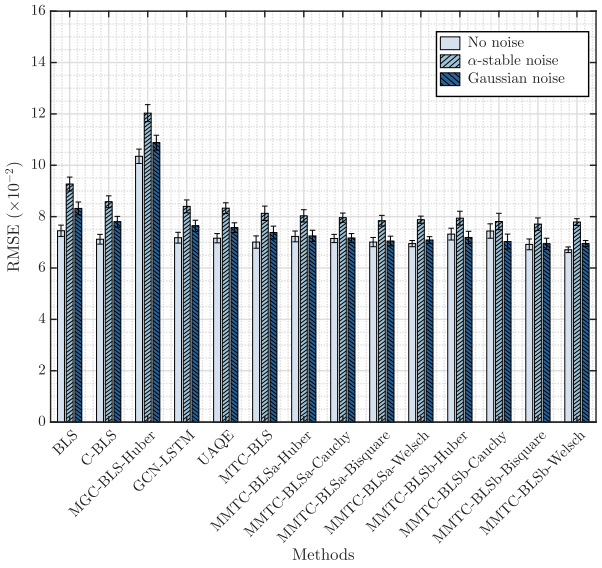
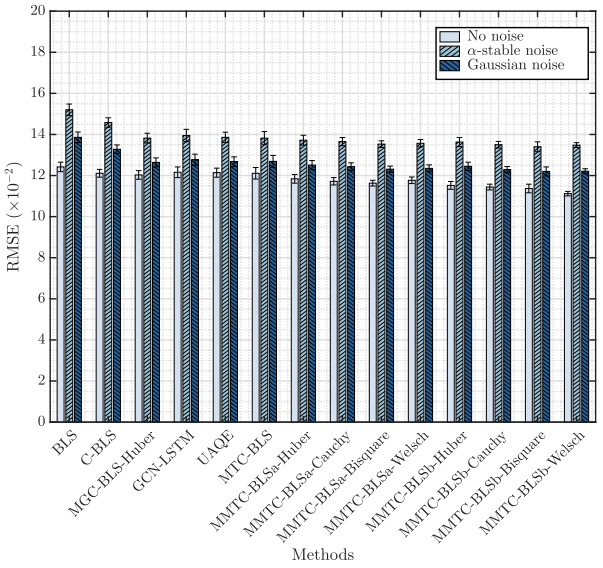


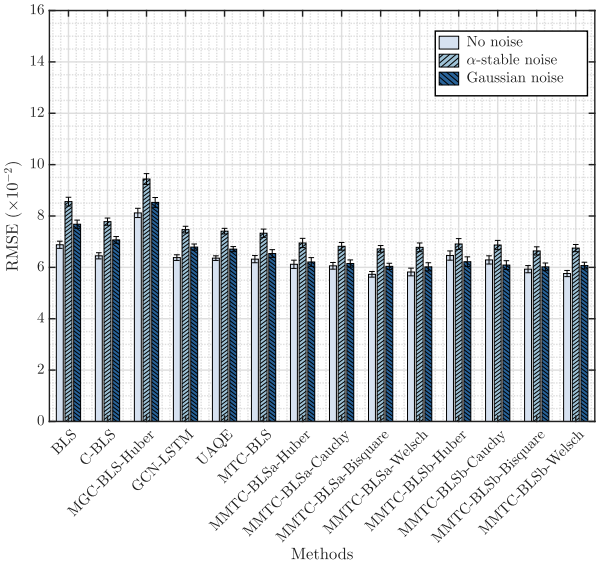
Fig. 5. Experimental results of the regression dataset with different noise types (Part I).



(e) Photovoltaic Power2

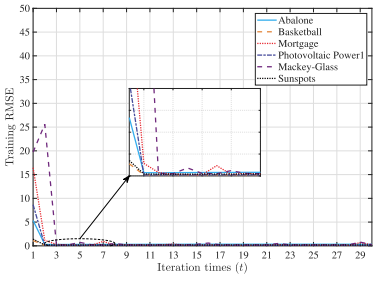


(f) SPARK-Raw

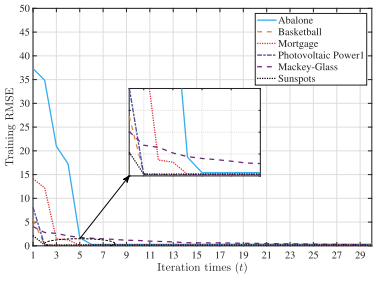


(g) Batch

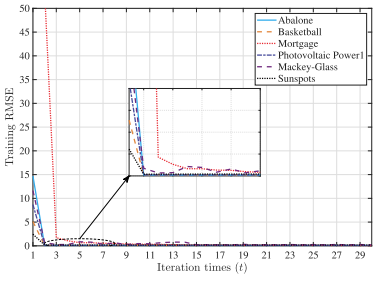
Fig. 5. Experimental results of the regression dataset with different noise types (Part II).



(a)



(b)



(c)

Fig. 6. The convergence curves of MTC-BLS, MMTC-BLSa-Huber and MMTC-BLSb-Huber. (a) MTC-BLS; (b) MMTC-BLSa-Huber; (c) MMTC-BLSb-Huber.

5.2.2. Comparative experiments with different noise contamination levels

Similarly, to further investigate the robustness of the proposed algorithms in regression tasks, different levels of noise are added to the inputs and outputs of the training samples following the approach in Chu et al. (2020), and comparative experiments are then conducted. Table 5 presents the RMSE±STD of different algorithms under various noise levels for each dataset. Meanwhile, Table 6 provides the corresponding RI values. Then the following observations can be drawn.

(1) As the noise level increases, the RMSE of all algorithms shows varying degrees of growth, yet the proposed methods consistently achieve better prediction accuracy. Furthermore, as indicated by Table 6, the proposed methods also attain relatively higher RI values, with a smoother decline in performance. This demonstrates that our proposed algorithms can effectively enhance the noise resistance of the models while also exhibiting improved robustness.

(2) As observed in Table 5, compared with the other methods, the RMSE of BLS and C-BLS increases more rapidly with rising noise levels, indicating that BLS and C-BLS are more sensitive to noise and outliers in the inputs and outputs of the training samples. This is consistent with the results shown in Table 6, where the RI values of BLS and C-BLS exhibit corresponding changes.

(3) On datasets such as Abalone and Basketball, although UQAE and GCN-LSTM perform reasonably well under low-noise conditions, their error growth rates are significantly higher than those of the proposed algorithms in high-noise regimes. As shown by the RI values in Table 6, the improved BLS methods proposed in this paper achieve the highest RI in the majority of cases. Further combined with the results in Table 5, it can be observed that our methods not only exhibit smaller relative performance degradation (i.e., higher RI), but also consistently maintain superior absolute accuracy. This demonstrates that the proposed approach is not merely noise-tolerant, but effectively achieves a balance between high accuracy and high robustness.

(4) From Table 6, it can be observed that in datasets such as Basketball and Photovoltaic Power1, the MMTC-BLS series algorithms often yield $RI > 100\%$. This occurs because small-sample or specific-distribution regression tasks are prone to overfitting, and moderate noise injection can act as data augmentation, increasing sample diversity and smoothing the decision boundary. By combining the MTC criterion with M-estimators, the proposed methods can effectively filter harmful noise while benefiting from noise-induced distribution perturbations, thereby improving generalization. Consequently, the experiments show that training with noisy data can outperform noise-free training.

5.2.3. Time cost comparison

To comprehensively address the evaluation requirements regarding computational efficiency, this section provides a detailed comparison of the average training time of the proposed algorithms and the benchmark methods across different regression datasets and noise levels. The experimental results are reported in Table S1 of Supplementary Materials. As shown in Table S1, the standard BLS achieves the fastest training speed due to its non-iterative pseudoinverse-based solution. The proposed MTC-BLS, MMTC-BLSa, and MMTC-BLSb introduce a fixed-point iteration mechanism, which results in increased training time compared with the standard BLS. This additional computational cost mainly arises from the matrix inversion operations involved in the iterative process. Compared with MGC-BLS-Huber, which also exhibits robustness, the proposed algorithms have time complexity of the same order of magnitude, demonstrating a reasonable computational cost. In fact, although the proposed algorithms introduce an iterative process, their computational efficiency consistently outperforms that of GCN-LSTM and UQAE. Moreover, in tasks with high feature dimensionality (i.e., the Batch dataset) and ultra-large-scale samples (e.g., the Photovoltaic Power2 dataset), the training time of the proposed algorithms does not exhibit exponential growth. In summary, although the fixed-point iteration makes the proposed algorithms slightly slower than the standard BLS, they achieve an effective balance between computational overhead and robustness gain.

5.3. Statistical analysis and testing

To further verify the robustness of different algorithms, we select the Huber function as the estimator and perform a post-hoc Friedman test based on the experimental results in Sections 5.1.2 and 5.2.2, and the results are shown in Table 7. It can be easily observed that the proposed MMTC-BLSb-Huber achieves the lowest average rank, significantly outperforming all comparison algorithms. It is followed by MMTC-BLSa-Huber and MTC-BLS. In contrast, the standard BLS has the highest rank, indicating that the introduction of the MTC criterion and the M-estimator can effectively improve the predictive performance of the algorithms. Moreover, the p-values between the three proposed algorithms and standard BLS or C-BLS are all far below the 0.05 significance level, indicating that the proposed algorithms have statistically significant superiority. Compared with advanced deep networks such as GCN-LSTM and UQAE, the proposed algorithms not only achieve higher rankings but also pass significance tests, which confirms that our algorithms are more competitive than general deep neural networks when handling datasets with complex noise.

5.4. Classification on image datasets

The experiments in Sections 5.1 to 5.3 fully validate the regression prediction capability and robustness of the three proposed algorithms. To further investigate their performance in classification tasks, this section conducts classification experiments on the MNIST (Lecun et al., 1998), NYU Object Recognition Benchmark (NORB) (LeCun et al., 2004), CIFAR-10 (Krizhevsky, 2009), and CIFAR-100 (Krizhevsky, 2009) datasets. Note that the Huber function is still chosen as the M-estimator for the MMTC-BLS variants in this section. The details of these image datasets are provided below.

1) MNIST: The MNIST dataset contains images from 10 classes, with 60,000 training images and 10,000 test images. Each image has a resolution of 28×28 pixels.

2) NORB: The NORB dataset is a coarse-grained image dataset, containing a total of 48,600 images from five classes: animals, humans, airplanes, trucks, and cars. Each image has a size of $2 \times 28 \times 28$ pixels. In our experiments, it is split into training and test sets at a 1:1 ratio.

3) CIFAR-10: The CIFAR-10 is one of the most popular image datasets in the field of computer vision. It contains 10 classes, 50,000 training images, and 10,000 test images. Each image has a size of 32×32 pixels.

3) CIFAR-100: The CIFAR-100 dataset belongs to the same collection as CIFAR-10 but is more challenging. It contains 100 classes, 50,000 training images, and 10,000 test images. Each image has a size of 32×32 pixels.

Table 8 records the accuracy and average training time of the different algorithms on these mainstream image datasets. The proposed MMTC-BLSb-Huber achieves the best classification performance on all image datasets. Notably, when handling the CIFAR-100 dataset with many classes and complex features, the traditional BLS is limited due to the lack of a robustness mechanism. In contrast, MMTC-BLSb-Huber, by combining the MTC criterion with the M-estimator, demonstrates stronger classification performance than GCN-LSTM and UQAE. This further validates the superiority of the strategy of "utilizing M-estimators as weighting factors" in processing high-dimensional image features, which effectively compensates for deviations arising during nonlinear mapping processes and enhances the model's robustness in discerning discriminative features. Moreover, although the introduction of the fixed-point iteration mechanism increases the training time compared with standard BLS, the overall computational cost of the proposed algorithms remains within a reasonable range compared with GCN-LSTM and UQAE.

Table 5
The experimental results (mean± STD) on regression datasets with different contamination rates (RMSE $\times 10^{-2}$).

Datasets	Noise rate	Methods													
		BLS	C-BLS	MGC-BLS -Huber	GCN-LSTM	UQAE	MTC-BLS	MMTC-BLSa -Huber	MMTC-BLSa -Cauchy	MMTC-BLSa -Bisquare	MMTC-BLSa -Welsch	MMTC-BLSb -Huber	MMTC-BLSb -Cauchy	MMTC-BLSb -Bisquare	MMTC-BLSb -Welsch
Abalone	0%	8.50±0.13	8.31±0.14	8.22±0.11	8.27±0.16	8.26±0.14	8.25±0.14	8.11±0.13	7.92±0.08	8.00±0.11	7.91±0.09	7.98±0.12	7.74±0.07	7.83±0.14	**7.72±0.08**
	10%	8.63±0.28	8.42±0.19	8.28±0.23	8.27±0.21	8.11±0.19	8.19±0.17	8.10±0.19	8.02±0.26	8.05±0.16	7.85±0.18	7.77±0.29	7.73±0.16	7.81±0.23	**7.68±0.13**
	20%	8.74±0.37	8.57±0.24	8.23±0.18	8.07±0.27	8.31±0.24	8.15±0.33	8.10±0.28	8.22±0.31	8.07±0.34	7.94±0.23	7.78±0.33	7.71±0.39	7.85±0.28	7.79±0.29
	30%	8.86±0.43	8.71±0.39	8.13±0.31	8.21±0.32	7.89±0.28	8.05±0.44	8.00±0.27	8.16±0.34	8.24±0.43	8.07±0.21	7.90±0.41	7.84±0.24	7.95±0.33	**7.76±0.31**
	40%	9.07±0.38	8.93±0.43	8.25±0.42	8.02±0.36	8.26±0.31	8.18±0.41	8.22±0.43	8.29±0.37	8.12±0.43	8.13±0.37	8.01±0.33	7.95±0.41	8.05±0.44	**7.88±0.42**
	50%	9.24±0.41	9.11±0.38	8.30±0.44	8.27±0.42	8.44±0.39	8.27±0.39	8.28±0.41	8.23±0.36	8.23±0.43	8.12±0.42	8.18±0.44	8.11±0.33	8.20±0.36	**7.99±0.43**
	60%	9.49±0.39	9.32±0.41	8.35±0.42	8.35±0.46	8.19±0.44	8.27±0.44	8.17±0.37	8.24±0.43	8.23±0.39	8.16±0.41	**7.89±0.44**	8.02±0.43	8.13±0.41	7.98±0.39
Basketball	0%	8.37±0.12	8.16±0.14	8.14±0.11	8.15±0.13	8.14±0.11	8.12±0.13	7.90±0.09	7.91±0.07	**7.77±0.08**	7.91±0.12	7.86±0.09	7.82±0.11	7.72±0.07	7.73±0.08
	10%	8.48±0.24	8.27±0.17	7.80±0.21	7.68±0.22	7.84±0.19	7.76±0.18	7.85±0.27	7.82±0.16	7.82±0.23	7.72±0.19	7.69±0.18	7.62±0.29	7.65±0.16	**7.60±0.24**
	20%	8.61±0.28	8.41±0.36	7.82±0.31	7.94±0.26	7.62±0.24	7.78±0.23	7.80±0.25	7.73±0.38	7.69±0.29	7.61±0.33	7.48±0.31	7.42±0.27	7.45±0.35	**7.39±0.21**
	30%	8.79±0.41	8.58±0.34	7.75±0.42	7.82±0.38	7.64±0.36	7.72±0.37	7.30±0.44	7.50±0.32	7.52±0.41	7.44±0.39	7.35±0.42	7.28±0.43	7.32±0.38	7.40±0.36
	40%	8.92±0.34	8.73±0.43	8.00±0.38	7.81±0.41	8.13±0.39	7.97±0.41	7.74±0.36	7.82±0.43	7.66±0.41	7.52±0.44	7.45±0.38	7.50±0.42	7.55±0.39	**7.41±0.43**
	50%	9.14±0.38	8.95±0.44	8.11±0.43	8.07±0.42	8.14±0.39	8.07±0.37	7.89±0.42	7.87±0.36	7.93±0.41	7.72±0.44	7.69±0.39	7.74±0.43	7.79±0.38	**7.64±0.41**
	60%	9.38±0.41	9.15±0.39	7.88±0.43	7.99±0.44	7.67±0.42	7.83±0.41	7.91±0.38	7.94±0.44	7.92±0.42	7.79±0.37	7.75±0.41	7.70±0.44	7.65±0.39	**7.59±0.43**
Mortgage	0%	4.95±0.07	4.81±0.09	4.78±0.06	4.79±0.08	4.77±0.07	4.76±0.08	4.12±0.04	4.28±0.06	4.27±0.09	4.22±0.03	**4.10±0.04**	4.25±0.08	4.24±0.07	4.15±0.03
	10%	5.53±0.21	5.34±0.16	6.04±0.23	6.03±0.24	5.93±0.19	5.99±0.19	4.60±0.16	4.25±0.18	4.70±0.14	**4.23±0.21**	4.31±0.18	4.36±0.24	4.33±0.17	4.29±0.13
	20%	6.09±0.27	5.78±0.31	6.47±0.24	6.29±0.31	6.57±0.28	6.42±0.34	5.11±0.38	4.27±0.29	4.22±0.33	4.03±0.26	4.11±0.24	4.06±0.37	3.96±0.31	**3.91±0.19**
	30%	6.57±0.41	6.18±0.36	6.53±0.44	6.61±0.39	6.36±0.34	6.48±0.31	5.42±0.33	4.94±0.42	5.14±0.39	4.68±0.35	**4.49±0.43**	4.54±0.31	4.44±0.44	4.61±0.34
	40%	7.08±0.38	6.61±0.43	6.38±0.39	6.26±0.42	6.37±0.37	6.31±0.41	6.07±0.44	5.40±0.41	4.70±0.42	4.35±0.44	4.30±0.33	4.32±0.41	4.25±0.38	**4.18±0.37**
	50%	7.56±0.41	7.11±0.38	6.51±0.44	6.47±0.43	6.39±0.41	6.45±0.37	5.58±0.42	5.00±0.43	5.38±0.39	5.08±0.41	4.90±0.33	4.85±0.44	4.80±0.36	**4.73±0.43**
	60%	8.09±0.43	7.62±0.41	6.65±0.39	6.67±0.45	6.73±0.41	6.60±0.44	6.06±0.42	6.17±0.37	5.95±0.41	5.19±0.43	5.00±0.36	4.95±0.39	4.85±0.44	**4.77±0.38**
Photovoltaic Power1	0%	9.15±0.23	8.98±0.17	8.95±0.19	8.94±0.16	8.93±0.14	8.92±0.16	8.73±0.14	8.63±0.12	8.58±0.14	8.66±0.11	8.74±0.13	8.71±0.19	8.55±0.21	8.63±0.14
	10%	9.28±0.35	9.07±0.29	8.95±0.24	8.71±0.31	9.07±0.28	8.89±0.31	8.70±0.33	8.96±0.37	8.43±0.32	8.58±0.28	8.48±0.41	8.38±0.34	8.41±0.39	**8.33±0.24**
	20%	9.43±0.41	9.22±0.44	9.12±0.34	9.19±0.38	9.01±0.32	9.10±0.41	8.77±0.43	8.60±0.38	9.01±0.44	8.67±0.41	8.62±0.39	8.57±0.43	8.53±0.36	**8.51±0.37**
	30%	9.57±0.37	9.38±0.42	9.12±0.39	9.01±0.41	9.19±0.36	9.10±0.44	9.04±0.41	8.79±0.33	8.77±0.44	8.72±0.39	8.59±0.42	8.68±0.36	8.62±0.43	8.65±0.38
	40%	9.78±0.43	9.59±0.41	9.17±0.44	9.31±0.43	8.97±0.38	9.13±0.38	9.04±0.42	9.04±0.37	9.06±0.41	8.79±0.43	8.74±0.44	8.68±0.41	8.64±0.39	**8.62±0.42**
	50%	9.94±0.44	9.75±0.39	9.18±0.41	8.94±0.45	9.12±0.41	9.12±0.43	8.69±0.44	8.90±0.42	8.93±0.38	8.81±0.37	8.77±0.39	8.72±0.44	8.69±0.42	**8.67±0.43**
	60%	10.11±0.42	9.92±0.44	9.31±0.41	9.37±0.46	9.19±0.42	9.28±0.39	9.22±0.43	8.94±0.41	9.03±0.37	8.94±0.44	**8.69±0.42**	8.74±0.39	8.77±0.43	8.81±0.41
Photovoltaic Power2	0%	7.45±0.22	7.12±0.19	10.35±0.28	7.18±0.21	7.16±0.18	7.01±0.24	7.23±0.21	7.15±0.16	7.01±0.18	6.95±0.12	7.32±0.23	7.44±0.28	6.92±0.21	**6.71±0.11**
	10%	7.62±0.31	7.28±0.24	10.58±0.36	7.46±0.28	7.16±0.24	7.31±0.33	7.38±0.31	7.42±0.22	7.21±0.26	7.12±0.24	7.54±0.28	7.66±0.34	7.18±0.29	**6.88±0.21**
	20%	8.12±0.41	7.41±0.33	10.74±0.41	7.62±0.32	7.76±0.28	7.68±0.39	7.86±0.42	7.31±0.34	7.42±0.36	7.63±0.31	7.89±0.38	7.59±0.41	7.12±0.32	**6.94±0.22**
	30%	8.31±0.38	7.62±0.41	11.72±0.44	7.41±0.36	7.27±0.31	7.34±0.41	8.12±0.45	7.76±0.38	7.81±0.41	7.26±0.33	7.71±0.42	8.02±0.44	7.43±0.39	**7.12±0.24**
	40%	8.93±0.44	7.54±0.39	11.08±0.48	7.31±0.41	7.61±0.38	7.46±0.44	8.19±0.48	7.48±0.42	7.91±0.44	7.46±0.38	8.02±0.44	7.64±0.41	7.24±0.36	**7.16±0.26**
	50%	8.91±0.46	7.73±0.42	11.96±0.51	7.64±0.44	7.49±0.39	7.64±0.46	8.46±0.52	8.71±0.44	8.36±0.48	7.56±0.41	8.55±0.48	8.18±0.46	**7.38±0.42**	7.61±0.28
	60%	9.62±0.48	9.24±0.44	11.04±0.53	7.81±0.46	7.74±0.41	7.66±0.48	8.72±0.54	8.69±0.46	7.83±0.44	7.41±0.42	7.91±0.46	8.42±0.48	**7.18±0.41**	7.22±0.33
SPARK-Raw	0%	12.42±0.23	12.11±0.19	12.03±0.21	12.16±0.26	12.14±0.22	12.11±0.28	11.84±0.21	11.72±0.18	11.63±0.14	11.77±0.16	11.52±0.19	11.44±0.14	11.37±0.21	**11.12±0.11**
	10%	12.86±0.31	12.63±0.24	12.42±0.33	12.35±0.31	12.59±0.28	12.47±0.36	12.04±0.28	12.12±0.24	11.84±0.21	12.18±0.23	11.73±0.26	11.58±0.21	11.82±0.26	**11.39±0.16**
	20%	13.33±0.34	13.31±0.28	12.46±0.38	12.74±0.34	12.24±0.31	12.49±0.39	12.12±0.32	12.72±0.28	12.76±0.24	12.46±0.28	11.96±0.31	11.83±0.24	11.84±0.32	**11.28±0.19**
	30%	14.22±0.38	13.43±0.32	12.64±0.41	13.14±0.38	12.88±0.34	13.01±0.42	13.48±0.36	12.87±0.32	13.38±0.28	12.52±0.31	12.63±0.34	12.44±0.28	12.38±0.36	**11.56±0.22**
	40%	13.12±0.41	13.88±0.36	13.32±0.44	13.51±0.41	14.07±0.38	13.79±0.44	13.62±0.41	13.46±0.36	13.26±0.32	12.66±0.34	12.52±0.38	12.18±0.32	12.54±0.41	**11.76±0.26**
	50%	15.34±0.44	13.22±0.39	13.23±0.48	14.11±0.44	13.97±0.41	14.11±0.48	13.38±0.44	13.06±0.39	12.44±0.36	14.58±0.38	12.52±0.41	13.26±0.36	**12.14±0.44**	12.33±0.29
	60%	14.94±0.48	14.66±0.42	14.01±0.51	14.98±0.48	14.86±0.44	14.71±0.51	14.82±0.48	13.71±0.42	14.09±0.39	13.82±0.41	12.36±0.44	12.38±0.39	**11.89±0.48**	12.96±0.33
Batch	0%	6.88±0.14	6.45±0.12	8.12±0.18	6.38±0.11	6.36±0.09	6.32±0.14	6.41±0.16	6.35±0.13	**6.02±0.11**	6.11±0.15	6.75±0.18	6.58±0.16	6.22±0.14	6.05±0.12
	10%	7.06±0.18	6.68±0.15	8.28±0.22	6.48±0.14	6.36±0.12	6.42±0.18	6.58±0.21	6.52±0.18	**6.14±0.16**	6.33±0.19	6.86±0.22	6.66±0.21	6.38±0.18	6.21±0.14
	20%	7.38±0.21	6.76±0.18	8.72±0.26	6.67±0.16	6.93±0.14	6.81±0.21	6.98±0.24	6.73±0.21	6.24±0.19	6.46±0.22	7.06±0.26	6.92±0.24	6.56±0.21	**6.22±0.16**
	30%	7.82±0.24	7.01±0.21	8.54±0.29	7.17±0.19	6.89±0.16	7.03±0.24	6.94±0.28	6.68±0.24	6.48±0.22	**6.31±0.26**	7.46±0.29	7.11±0.28	6.54±0.24	6.88±0.19
	40%	7.76±0.28	7.32±0.24	9.06±0.33	6.71±0.21	6.86±0.18	6.78±0.28	7.66±0.31	6.81±0.28	6.54±0.26	6.94±0.29	7.18±0.32	7.31±0.31	6.58±0.28	**6.24±0.22**
	50%	8.32±0.32	7.11±0.28	9.68±0.36	7.63±0.24	7.54±0.21	7.63±0.32	6.91±0.34	6.98±0.32	7.46±0.29	6.71±0.32	7.04±0.36	7.23±0.34	7.02±0.32	**6.31±0.26**
	60%	7.64±0.36	7.82±0.31	10.08±0.41	7.72±0.26	7.79±0.24	7.64±0.36	7.44±0.38	7.02±0.36	7.24±0.32	7.86±0.36	7.02±0.41	7.66±0.38	7.16±0.36	**6.84±0.31**

Table 6
The robustness index (RI) on regression datasets with different contamination rates (%).

Datasets	Noise rate	Methods													
		BLS	C-BLS	MGC-BLS -Huber	GCN-LSTM	UQAE	MTC-BLS	MMTC-BLSa -Huber	MMTC-BLSa -Cauchy	MMTC-BLSa -Bisquare	MMTC-BLSa -Welsch	MMTC-BLSb -Huber	MMTC-BLSb -Cauchy	MMTC-BLSb -Bisquare	MMTC-BLSb -Welsch
Abalone	10%	98.49	98.69	99.28	100.00	101.85	100.73	100.12	98.75	99.38	100.76	**102.70**	100.13	100.26	100.52
	20%	97.25	92.13	99.88	102.48	99.40	101.23	100.12	96.35	99.13	91.34	**102.57**	100.39	99.75	96.98
	30%	95.94	90.42	101.44	100.73	104.69	**102.48**	101.38	91.45	90.09	91.66	101.01	89.72	91.47	96.74
	40%	87.72	84.88	99.64	103.12	100.00	**100.86**	98.66	93.18	98.52	87.02	99.63	97.36	97.27	93.80
	50%	93.41	91.22	99.04	100.00	97.87	**99.76**	83.09	86.65	97.21	80.71	97.56	90.42	87.88	96.62
	60%	89.57	89.16	98.44	99.04	100.86	99.76	99.27	**93.73**	97.21	96.94	101.14	96.51	96.31	96.74
Basketball	10%	98.70	98.67	104.36	106.12	103.83	**104.64**	100.64	101.15	99.36	102.46	102.21	102.62	100.92	101.71
	20%	97.21	97.03	104.09	102.64	106.82	104.37	101.28	102.33	101.04	103.94	105.08	**105.39**	103.62	104.60
	30%	95.22	92.20	105.03	104.22	106.54	105.18	108.22	105.47	103.32	106.32	106.94	**107.42**	105.46	104.46
	40%	93.83	93.47	101.75	104.35	100.12	101.88	102.07	101.15	101.44	105.19	**105.50**	104.27	102.25	104.32
	50%	91.58	91.17	100.37	100.99	100.00	100.62	100.13	100.51	97.98	**102.46**	102.21	101.03	99.10	101.18
	60%	89.23	89.18	103.30	102.00	106.13	**103.70**	99.87	99.62	98.11	101.54	101.42	101.56	100.92	101.84
Mortgage	10%	89.51	96.59	79.14	79.44	80.44	79.47	95.37	**100.71**	90.85	99.76	99.27	98.15	97.92	96.74
	20%	81.28	92.32	73.88	76.15	72.60	74.14	80.63	100.23	101.18	104.71	99.76	104.68	107.07	106.14
	30%	75.34	88.75	73.20	72.47	75.00	73.46	91.76	95.32	89.89	90.17	94.91	89.47	**95.50**	90.02
	40%	87.46	91.44	92.64	76.52	74.88	75.44	67.87	90.68	90.85	97.01	95.35	98.38	**99.76**	85.39
	50%	90.66	80.43	73.43	74.03	74.65	73.80	81.10	82.47	79.37	83.07	94.04	**95.72**	88.33	87.74
	60%	85.49	83.65	81.02	71.81	70.88	72.12	83.23	86.29	**92.62**	81.31	82.00	85.86	87.42	87.00
Photovoltaic Power1	10%	98.61	99.01	97.49	102.64	98.46	100.34	97.87	96.32	101.78	**100.93**	103.07	103.94	101.66	103.60
	20%	97.03	97.40	98.14	97.28	99.11	98.02	91.80	100.35	95.23	99.88	101.39	**101.63**	100.23	101.41
	30%	95.73	95.74	**98.14**	99.22	97.17	98.02	94.69	92.80	97.83	99.31	101.75	100.35	99.19	91.03
	40%	86.24	87.44	97.60	96.03	99.55	**97.70**	86.95	85.28	94.70	85.32	100.00	100.35	98.96	88.88
	50%	89.53	88.56	97.49	100.00	97.92	97.81	86.35	90.84	96.08	87.30	99.66	99.89	98.39	**95.68**
	60%	92.52	77.75	96.13	95.41	97.17	96.12	94.69	90.46	95.02	96.87	**100.58**	99.66	85.33	93.00
Photovoltaic Power2	10%	97.77	97.80	97.83	96.25	100.00	95.90	97.97	96.36	97.23	97.61	**98.79**	97.00	96.38	97.53
	20%	91.75	96.09	96.37	94.23	92.27	91.28	92.81	97.81	94.47	91.09	92.77	**97.89**	97.19	96.69
	30%	89.65	93.44	88.31	96.90	98.49	95.50	89.04	92.14	89.76	**95.73**	94.94	92.77	93.14	94.24
	40%	83.43	94.43	93.41	98.22	94.09	93.97	88.28	95.59	88.62	93.16	91.27	**97.38**	95.58	93.72
	50%	83.61	92.11	86.54	93.98	95.59	91.75	85.46	82.09	83.85	91.93	85.71	90.95	**93.77**	88.17
	60%	77.44	77.06	93.75	91.93	92.51	91.51	82.91	82.28	89.53	93.79	92.54	88.36	**96.38**	92.94
SPARK-Raw	10%	96.58	95.88	96.86	98.46	96.43	97.11	98.34	96.70	98.23	96.63	98.21	**98.79**	96.19	97.63
	20%	93.17	90.98	96.55	95.45	99.18	96.96	97.69	92.14	91.14	94.46	96.32	96.70	96.03	**98.58**
	30%	87.34	90.17	95.17	92.54	94.25	93.08	87.83	91.06	86.92	94.01	91.21	91.96	91.84	**96.19**
	40%	94.66	87.18	90.32	90.01	86.28	87.82	86.93	87.07	87.71	92.97	92.01	93.92	90.67	**94.56**
	50%	80.96	91.60	90.93	86.18	86.90	85.83	88.49	89.74	93.49	80.73	92.01	86.27	**93.66**	90.19
	60%	83.13	82.61	85.87	81.17	81.70	82.32	80.00	85.48	82.54	85.17	93.20	92.41	**95.63**	85.80
Batch	10%	97.45	96.56	98.07	98.46	100.00	98.44	97.42	97.39	**98.05**	96.52	98.40	98.80	97.49	97.42
	20%	93.22	95.41	93.12	95.65	91.77	92.80	91.83	94.35	96.47	94.58	95.61	95.09	95.11	**97.27**
	30%	87.98	92.01	95.08	88.98	92.31	89.90	92.36	95.06	92.90	**96.83**	90.48	92.55	95.11	88.23
	40%	88.66	88.11	89.62	95.08	92.71	93.22	83.68	93.25	92.05	88.04	94.01	90.01	94.53	**96.96**
	50%	82.69	90.72	83.88	83.62	84.35	82.83	92.76	90.97	80.70	91.06	**95.88**	91.01	88.86	95.88
	60%	90.05	82.48	80.56	82.64	81.64	82.72	86.16	90.46	83.15	77.73	**96.15**	85.90	86.87	88.45

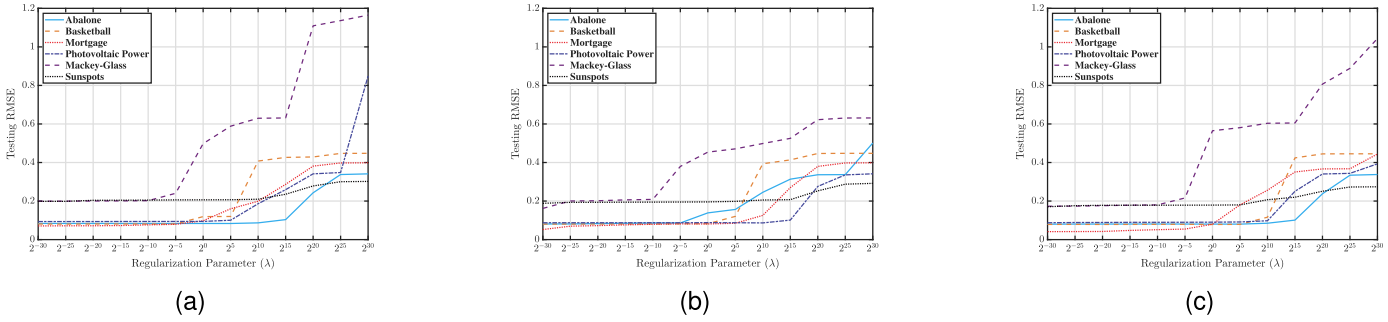


Fig. 7. The sensitivity analysis of the regularization parameter λ. (a) MTC-BLS; (b) MMTC-BLSa-Huber; (c) MMTC-BLSb-Huber.

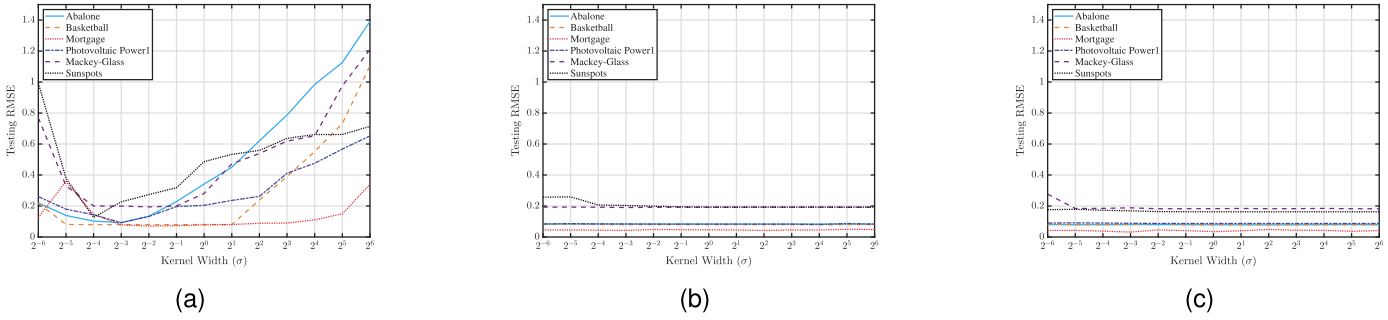


Fig. 8. The sensitivity analysis of the kernel width σ with noise-free. (a) MTC-BLS; (b) MMTC-BLSa-Huber; (c) MMTC-BLSb-Huber.

Table 7
Friedman Post-hoc test results of the proposed methods and other algorithms.

Algorithms	BLS	C-BLS	MGC-BLS -Huber	GCN-LSTM	UQAE	MTC-BLS	MMTC-BLSa -Huber	MMTC-BLSb -Huber
Mean Rank	7.47	5.84	5.49	4.57	4.12	3.40	3.12	1.98
MTC-BLS	0.00e+00	2.37e-08	1.73e-06	7.51e-03	1.02e-01	–	5.13e-01	1.13e-03
MMTC-BLSa	4.45e-10	5.41e-08	8.75e-04	2.19e-02	5.13e-01	5.13e-01	–	9.31e-03
MMTC-BLSb	0.00e+00	0.00e+00	8.88e-16	3.06e-09	1.00e-06	1.13e-03	9.31e-03	–

Table 8
Classification accuracy on image datasets.

Datasets	Algorithms	MNIST	NORB	CIFAR-10	CIFAR-100
BLS	Accuracy (%)	97.37	83.96	83.27	71.27
	Traing time (s)	27.34	25.17	41.91	33.79
C-BLS	Accuracy (%)	98.17	86.64	82.97	79.24
	Traing time (s)	55.02	60.22	88.99	93.44
MGC-BLS-Huber	Accuracy (%)	98.84	89.49	84.92	74.16
	Traing time (s)	58.11	53.38	81.12	70.69
GCN-LSTM	Accuracy (%)	98.92	89.53	88.42	81.34
	Traing time (s)	412.33	389.16	987.21	1204.58
UQAE	Accuracy (%)	98.56	89.82	88.74	79.41
	Traing time (s)	356.47	315.79	892.33	1056.82
MTC-BLS	Accuracy (%)	99.08	90.34	89.23	78.46
	Traing time (s)	54.89	68.12	75.34	79.21
MMTC-BLSa-Huber	Accuracy (%)	99.14	90.89	89.56	82.37
	Traing time (s)	65.41	53.97	73.18	96.54
MMTC-BLSb-Huber	Accuracy (%)	**99.27**	**91.62**	**90.84**	**83.19**
	Traing time (s)	62.33	59.46	91.27	104.21

5.5. Convergence analysis

In the Section 4.2, we explored the convergence properties of the proposed improved algorithms from a theoretical perspective. In this subsection, we randomly select 6 datasets to further conduct empirical analyses on the convergence of the proposed algorithms to ensure consistency between theory and practice. To this end, we calculated the corresponding training RMSE values for MTC-BLS, MMTC-BLSa, and MMTC-BLSb at each of 30 iterations, and plotted the convergence curves as shown in Fig. 6. Note that the Huber function is used as the M-estimator in the analysis of this subsection. As shown in Fig. 6, with the increase in the number of iterations, the RMSE values during the training phase decrease rapidly and gradually become stable, indicating that the three types of improved robust BLS algorithms proposed in this paper achieve stable convergence during the training process. Moreover, the RMSE values of all algorithms stabilize within 10 iterations across all datasets. Therefore, in practical applications, it is unnecessary to set an excessively large maximum number of iterations, which to some extent reduces the computational cost of the algorithms.

5.6. Parameters sensitivity analysis

For the three types of novel robust BLS proposed in this paper (*i.e.*, MTC-BLS, MMTC-BLSa, and MMTC-BLSb), several important hyperparameters are involved, including the regularization parameter λ and the kernel width σ. To examine the effect of hyperparameter settings on algorithm performance, a series of parameter sensitivity experiments are conducted in this subsection. This section uses the same datasets and M-estimator as the Section 5.5.

5.6.1. Sensitivity analysis of the regularization parameter λ

To avoid interference from other factors, each experiment adopts the optimal network structure corresponding to its dataset, while fixing the kernel width $\sigma = 0.1$. The effect of varying the regularization parameter

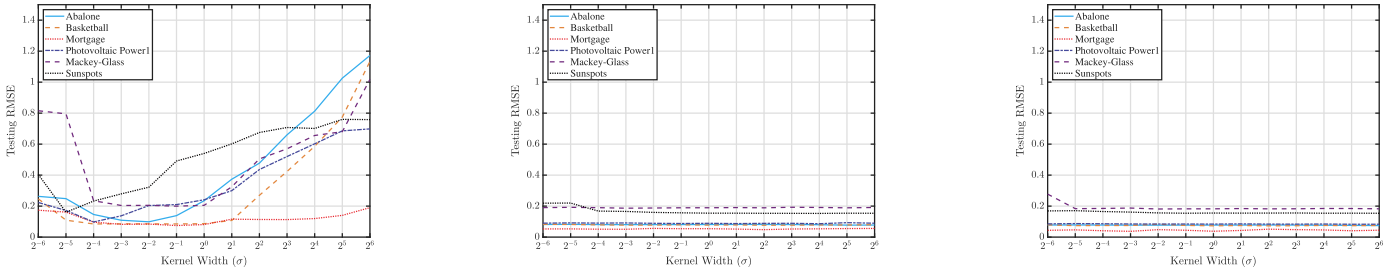


Fig. 9. The sensitivity analysis of the kernel width σ with 20% noise. (a) MTC-BLS; (b) MMTC-BLSa-Huber; (c) MMTC-BLSb-Huber.

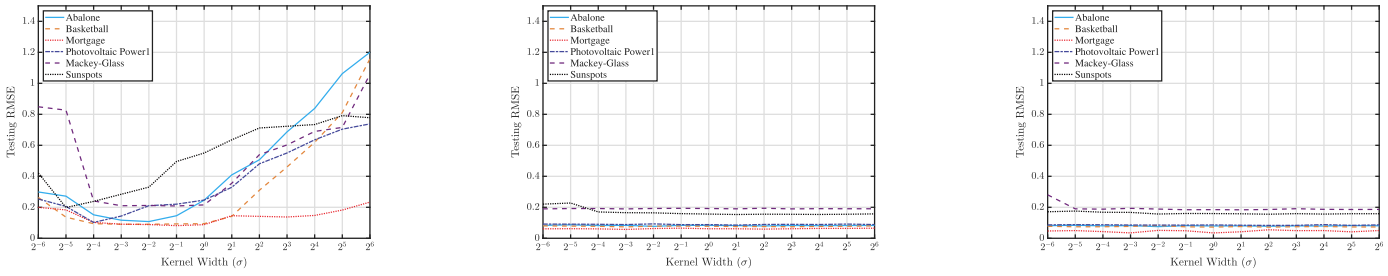


Fig. 10. The sensitivity analysis of the kernel width σ with 40% noise. (a) MTC-BLS; (b) MMTC-BLSa-Huber; (c) MMTC-BLSb-Huber.

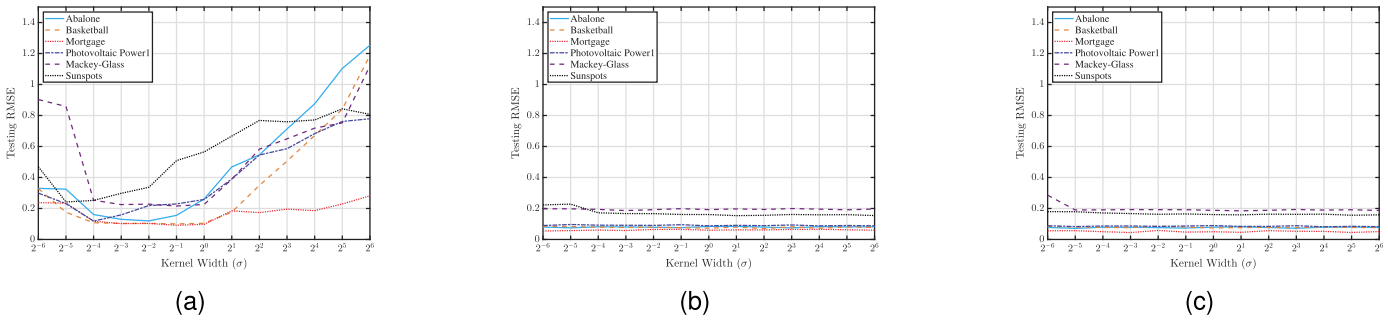


Fig. 11. The sensitivity analysis of the kernel width σ with 60% noise. (a) MTC-BLS; (b) MMTC-BLSa-Huber; (c) MMTC-BLSb-Huber.

λ within the range $\{2^{-30}, 2^{-25}, 2^{-20}, \ldots, 2^{20}, 2^{25}, 2^{30}\}$ on the prediction accuracy of the algorithms is then examined. The results are shown in Fig. 7. As shown in the figure, when the value of the regularization parameter λ is smaller than 2^{-10}, the variation in RMSE across all algorithms and datasets is very mild. In other words, under this condition, the choice of the regularization parameter has only a negligible effect on the predictive performance of the algorithms. Whereas once the regularization parameter exceeds 2^{-10}, the performance of all algorithms drops sharply as the parameter value increases. These results indicate that the regularization parameter λ can be easily determined by selecting a sufficiently small positive value close to zero, which aligns with the original design intention of our algorithms.

5.6.2. Sensitivity analysis of the kernel width σ

Similarly, each experiment adopts the optimal network structure corresponding to its dataset to avoid interference from unrelated factors. MMTC-BLSa and MMTC-BLSb mitigate the impact of kernel width selection bias by introducing an M-estimator in our study. In this case, the regularization parameter λ is fixed at 2^{-30}, and the effect of varying the kernel width σ within $\{2^{-6}, 2^{-5}, 2^{-4}, \ldots, 2^{4}, 2^{5}, 2^{6}\}$ under different noise levels on algorithm performance is examined. The results are presented in Figs. 8 to 11. It can be readily observed that the performance of MTC-BLS deteriorates sharply as the kernel width deviates from its optimal value, and this trend is particularly pronounced when the deviation is large. For MMTC-BLSa-Huber and MMTC-BLSb-Huber, the performance remains stable in most cases, with only slight fluctuations when the kernel width takes relatively small values, indicating that the introduction of the M-estimator significantly improves the performance stability of entropy-based BLS by confining performance variations to a narrow range, thereby eliminating the need to carefully tune the kernel width to guarantee predictive performance; this observation is consistent with the results reported by Zhao et al. (2025). For MTC-BLS, the theoretical analyses in Theorems 1 and 2 indicate that σ must satisfy a certain lower-bound condition (i.e., $\sigma \geq \max\{\sigma^{\star}, \sigma^{*}\}$) to guarantee the contraction property of the fixed-point iteration and thus ensure algorithm convergence. In practice, this implies that one should start with relatively large values of σ and gradually decrease it to search for an optimal robustness interval. Further combined with the results shown in Figs. 8a to 11a, the optimal σ for MTC-BLS typically lies in the range of 2^{-5} to 2^{-1}. Therefore, for MTC-BLS, a relatively small σ can be chosen to pursue high accuracy in noise-free or low-noise environments, while σ should be appropriately increased as the noise level rises to cover a wider error distribution and maintain convergence stability.

6. Conclusion

In this paper, to address the limitation that BLS and some of its improved robust variants can only handle noise in the outputs of training samples and cannot effectively process noise in the input data, a maximum total correntropy-based BLS (MTC-BLS) is proposed. It combines the advantages of the TLS method and the MCC criterion, enabling it to effectively handle both input and output noise. Meanwhile, to reduce the dependence of MTC-BLS on the selection of kernel parameters, two novel robust BLS variants are further proposed, namely MMTC-BLSa and MMTC-BLSb. The former trains the model by incorporating both the M-estimator and the MTC criterion into the construction of the objective function, ensuring model robustness. The latter uses the M-estimator as a weighting factor within the MTC criterion during training to compensate for model errors caused by deviations in kernel parameter selection, thereby effectively mitigating the negative impact of kernel parameters deviating from their optimal values. Comparative experiments on time series datasets, regression datasets and image datasets demonstrate the effectiveness and superiority of our algorithms.

However, the optimal network structure was determined using grid search in the experimental section, which is time-consuming. In future work, we plan to explore the use of efficient heuristic optimization algorithms, such as ant colony optimization and genetic algorithms, to determine the optimal parameters more efficiently. Furthermore, this paper focuses on applying the proposed methods to regression tasks. In fact, extending the improved algorithms to classification tasks will also be a key focus of our future work.

CRediT authorship contribution statement

Tao Chen: Writing – original draft, Methodology, Formal analysis; **Xi Chen:** Writing – review & editing, Resources, Data curation; **Wei Li:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was partially supported by National Natural Science Foundation of China (72374077).

Appendix A. KKT derivation of MMTC-BLSa

Eq. (16) presents the Lagrangian function of MMTC-BLSa:

$$\mathcal{J}_{MA} = \sum_{i=1}^{N} \exp\left(-\frac{\|\boldsymbol{A}_i\boldsymbol{W}-\boldsymbol{Y}_i\|_2^2}{2\sigma^2\|\bar{\boldsymbol{W}}\|_2^2}\right) - \sum_{i=1}^{N}\rho(e_i) - \frac{\lambda}{2}\|\boldsymbol{W}\|_2^2 - \sum_{i=1}^{N}\eta_i\left(\boldsymbol{Y}_i - \boldsymbol{A}_i\boldsymbol{W} - e_i\right) \tag{A.1}$$

where the constraint is given by $\boldsymbol{Y}_i - \boldsymbol{A}_i\boldsymbol{W} - e_i = 0$, and η_i denotes the corresponding Lagrange multiplier. We perform optimization on Eq. (A.2) through the following steps.

Step 1. Obtain the KKT conditions of the objective function: Take partial derivatives of $\mathcal{J}_{MA}$ with respect to $\boldsymbol{W}$, e_i, and η_i, respectively, and set them to zero:

$$\frac{\partial\mathcal{J}_{MA}}{\partial\boldsymbol{W}} : -\frac{1}{\sigma^2\|\boldsymbol{W}\|_2^2}\left(\boldsymbol{A}^T\boldsymbol{\Lambda}_M(\boldsymbol{A}\boldsymbol{W}-\boldsymbol{Y}) - s_M\boldsymbol{W}\right) - \lambda\boldsymbol{W} + \sum_{i=1}^{N}\eta_i\boldsymbol{A}_i^T = 0 \tag{A.2}$$

$$\frac{\partial\mathcal{J}_{MA}}{\partial e_i} : -\frac{\partial\rho(e_i)}{\partial e_i} + \eta_i = 0 \tag{A.3}$$

$$\frac{\partial\mathcal{J}_{MA}}{\partial\eta_i} : \boldsymbol{Y}_i - \boldsymbol{A}_i\boldsymbol{W} - e_i = 0 \tag{A.4}$$

Step 2. Obtain the relationship between the Lagrange multipliers and the M-estimator influence function: From Eq. (A.3), one has:

$$\eta_i = \frac{\partial\rho(e_i)}{\partial e_i} \triangleq \phi(e_i) \tag{A.5}$$

where, $\phi(e_i)$ denotes the influence function (or score function) in M-estimation. This equation is a direct consequence of the KKT conditions and does not require any additional assumptions.

Step 3. Introduce the weight function of the M-estimator: In the iterative reweighted least squares (IRLS) algorithm for M-estimation, the weight function is typically defined as:

$$\theta_i = \frac{\phi(e_i)}{e_i} = \frac{\partial\rho(e_i)/\partial e_i}{e_i} \tag{A.6}$$

This definition is clearly established in standard textbooks (Guo & Xu, 2023; Huber, 1981; Zhao et al., 2025). Its purpose is to transform the gradient condition $\phi(e_i) = 0$ into the weighted zero-mean condition $\theta_i e_i = 0$, thereby making the M-estimation problem equivalent to a weighted least squares problem.

Step 4. Combine Eqs. (A.5) and (A.6): From (A.5), we have $\eta_i = \phi(e_i)$. Substituting this into (A.6) yields:

$$\theta_i = \frac{\eta_i}{e_i} \quad \Rightarrow \quad \eta_i = \theta_i e_i \tag{A.7}$$

In matrix form, Eq. (A.7) can be written as:

$$\boldsymbol{\eta} = \boldsymbol{\theta}\boldsymbol{e}, \quad \boldsymbol{\theta} = \mathrm{diag}(\theta_1, \theta_2, \ldots, \theta_N) \tag{A.8}$$

Then, Eq. (18) is obtained.

Step5. Obtain the expression for the output weights: Substituting (A.8) and (A.4) (i.e., $e = \boldsymbol{Y} - \boldsymbol{A}\boldsymbol{W}$) into Eq. (A.4), and performing the matrix operations, one obtains Eq. (19) as follows:

$$\boldsymbol{W} = \left(\boldsymbol{A}^T\boldsymbol{\Lambda}_{MA}\boldsymbol{A} + \gamma_M\boldsymbol{I}\right)^{-1}\boldsymbol{A}^T\boldsymbol{\Lambda}_{MA}\boldsymbol{Y}$$

Supplementary material

Supplementary material associated with this article can be found in the online version at 10.1016/j.neunet.2026.108917.

References

Bingham, E., & Hyvarinen, A. (2000). A fast fixed-point algorithm for independent component analysis of complex valued signals. *International Journal of Neural Systems*, *10*(01), 1–8. https://doi.org/10.1142/S0129065700000028

Ceni, A. (2025). Random orthogonal additive filters: A solution to the vanishing/exploding gradient of deep neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *36*(6), 10794–10807. https://doi.org/10.1109/TNNLS.2025.3538924

Chen, B., Xing, L., Wang, X., Qin, J., & Zheng, N. (2018). Robust learning with kernel mean p -power error loss. *IEEE Transactions on Cybernetics*, *48*(7), 2101–2113. https://doi.org/10.1109/TCYB.2017.2727278

Chen, C. L. P., & Liu, Z. (2018). Broad learning system: An effective and efficient incremental learning system without the need for deep architecture. *IEEE Transactions on Neural Networks and Learning Systems*, *29*(1), 10–24. https://doi.org/10.1109/TNNLS.2017.2716952

Chu, F., Liang, T., Chen, C. L. P., Wang, X., & Ma, X. (2020). Weighted broad learning system and its application in nonlinear industrial process modeling. *IEEE Transactions on Neural Networks and Learning Systems*, *31*(8), 3017–3031. https://doi.org/10.1109/TNNLS.2019.2935033

Clette, F., & Lefèvre, L. (2015). Silso sunspot number v2.0. Published by WDC SILSO - Royal Observatory of Belgium (ROB). https://doi.org/10.24414/qnza-ac80

Fouad, M. M., Dansereau, R. M., & Whitehead, A. D. (2010). Image registration under local illumination variations using robust bisquare m-estimation. In *2010 IEEE International conference on image processing* (pp. 917–920). https://doi.org/10.1109/ICIP.2010.5651267

Fu, K., Li, H., & Shi, X. (2024). An encoder-decoder architecture with Fourier attention for chaotic time series multi-step prediction. *Applied Soft Computing*, *156*, 111409. https://doi.org/10.1016/j.asoc.2024.111409

Gunasekaran, S., Kembay, A., Zhu, H. L. J., Perrinet, L., Kavehei, O., & Eshraghian, J. (2025). A predictive approach to enhance time-series forecasting. *Nature Communications*, *16*, 8645. 10./s41467-025-63786-4

Guo, W., & Xu, T. (2023). M-estimator-based robust broad learning system. *Control and Decision*, *38*(4), 1039–1046. https://doi.org/10.13195/j.kzyjc.2021.1479

Guo, Y. (2026). Multimodal spatio-temporal fusion: A generalizable GCN-LSTM with attention framework for urban application. *Information Fusion*, *131*, 104164. https://doi.org/10.1016/j.inffus.2026.104164

Hale, E. T., Yin, W., & Zhang, Y. (2008). Fixed-point continuation for ℓ_1-minimization: Methodology and convergence. *SIAM Journal on Optimization*, *19*(3), 1107–1130. https://doi.org/10.1137/070698920

He, Y., Wang, F., Wang, S., Ren, P., & Chen, B. (2019). Maximum total correntropy diffusion adaptation over networks with noisy links. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *66*(2), 307–311. https://doi.org/10.1109/TCSII.2018.2853653

Heravi, A. R., & Hodtani, G. A. (2017). A new robust fixed-point algorithm and its convergence analysis. *Journal of Fixed Point Theory and Applications*, *19*(4), 3191–3215.

Hou, X., Zhao, H., Long, X., & Jin, W. (2022). The kernel recursive maximum total correntropy algorithm. *IEEE Transactions on Circuits and Systems II: Express Briefs*, *69*(12), 5139–5143. https://doi.org/10.1109/TCSII.2022.3202800

Huber, P. J. (1981). Robust statistics. Wiley series in probability and statistics. New York: John Wiley & Sons, Inc. https://doi.org/10.1002/0471725250

Kelly, M., Longjohn, R., & Nottingham, K.. The UCI machine learning repository. Accessed: 2026-01-10. https://archive.ics.uci.edu.

Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical Report TR-2009. Department of Computer Science, University of Toronto Toronto, Ontario, Canada.

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278–2324. https://doi.org/10.1109/5.726791

LeCun, Y., Huang, F. J., & Bottou, L. (2004). Learning methods for generic object recognition with invariance to pose and lighting. In *Proceedings of the 2004 IEEE computer society conference on computer vision and pattern recognition, 2004. CVPR 2004.* (pp. II–104 Vol.2). (*vol. 2*). https://doi.org/10.1109/CVPR.2004.1315150

Lee, C.-C., Chiang, Y.-C., Shih, C.-Y., & Tsai, C.-L. (2009). Noisy time series prediction using M-estimator based robust radial basis function neural networks with growing and pruning techniques. *Expert Systems with Applications*, *36*(3, Part 1), 4717–4724. https://doi.org/10.1016/j.eswa.2008.06.017

Li, H., Gao, Y., Guo, X., & Ou, S. (2025). Variable-step-size generalized maximum correntropy affine projection algorithm with sparse regularization term. *Electronics*, *14*(2). https://doi.org/10.3390/electronics14020291

Li, W., & Swetits, J. J. (1998). The linear l1 estimator and the huber m-estimator. *SIAM Journal on Optimization*, *8*(2), 457–475. https://doi.org/10.1137/S1052623495293160

Li, Z. (2025). Sparse broad learning system via sequential threshold least-squares for nonlinear system identification under noise. arXiv preprint arXiv:2511.18081, .

Mallea, M., Nebot, Á., & Mugica, F. (2026). Advancing broad learning through structured feature generation. *Expert Systems with Applications*, *299*, 129948. https://doi.org/10.1016/j.eswa.2025.129948

Peng, C., & ChunHao, D. (2022). Monitoring multi-domain batch process state based on fuzzy broad learning system. *Expert Systems with Applications*, *187*, 115851. https://doi.org/10.1016/j.eswa.2021.115851

Polyanskiy, Y., & Wu, Y. (2025). Information theory: From coding to learning. Cambridge University Press.

Principe, J. C. (2010). Information theoretic learning: Renyi's entropy and kernel perspectives. Springer Science & Business Media.

Qin, A., Hu, Q., Zhang, Q., & Mao, H. (2025). A partial domain adaptation broad learning system for machinery fault diagnosis. *Measurement*, *243*, 116437. https://doi.org/10.1016/j.measurement.2024.116437

Rasch, M. J., Carta, F., Fagbohungbe, O., & Gokmen, T. (2024). Fast and robust analog in-memory deep neural network training. *Nature Communications*, *15*, 7133. https://doi.org/10.1038/s41467-024-51221-z

Rousseeuw, P. J., & Leroy, A. M. (1987). Robust regression and outlier detection. USA: John Wiley & Sons, Inc.

Sievers, J., Blank, T., Bischof, S., & Simon, F. (2026). Spark-raw: Unprocessed high-resolution energy data from a sustainable industrial production area in karlsruhe. 37.12.02; LK 01. https://doi.org/10.35097/8efkkn0g9pcng5yd

Wang, F., He, Y., Wang, S., & Chen, B. (2017). Maximum total correntropy adaptive filtering against heavy-tailed noises. *Signal Processing*, *141*, 84–95. https://doi.org/10.1016/j.sigpro.2017.05.029

Wang, J., Xie, F., Nie, F., & Li, X. (2024). Generalized and robust least squares regression. *IEEE Transactions on Neural Networks and Learning Systems*, *35*(5), 7006–7020. https://doi.org/10.1109/TNNLS.2022.3213594

Wang, Y., Wang, L., & Chen, T. (2026). Broad learning system via adaptive maximum weighted correntropy. *Neural Networks*, *193*, 108032. https://doi.org/10.1016/j.neunet.2025.108032

Xie, Y., Li, Y., Gu, Y., Cao, J., & Chen, B. (2020). Fixed-point minimum error entropy with fiducial points. *IEEE Transactions on Signal Processing*, *68*, 3824–3833. https://doi.org/10.1109/TSP.2020.3001404

Xu, X., Bao, S., Shao, H., & Shi, P. (2024). A multi-sensor fused incremental broad learning with D-S theory for online fault diagnosis of rotating machinery. *Advanced Engineering Informatics*, *60*, 102419. https://doi.org/10.1016/j.aei.2024.102419

Yin, Y., Long, Z., Jin, S., Li, Y., Wang, F., & Xu, X. (2025). Multiphysics coupling AI prediction method for thermomechanical behavior of steel ladle linings. *Knowledge-Based Systems*, *330*, 114495. https://doi.org/10.1016/j.knosys.2025.114495

Yuan, C., Zhou, C., Peng, J., & Li, H. (2024). Mixture correntropy-based robust distance metric learning for classification. *Knowledge-Based Systems*, *295*, 111791. https://doi.org/10.1016/j.knosys.2024.111791

Yuen, K.-V., & Kuok, S.-C. (2025). Telescopic broad bayesian learning for big data stream. *Computer-Aided Civil and Infrastructure Engineering*, *40*(1), 33–53. https://doi.org/10.1111/mice.13305

Zhang, S., Liu, Z., & Chen, C. L. P. (2022). Broad learning system based on the quantized minimum error entropy criterion. *Science China Information Sciences*, *65*(12), 222203.

Zhang, T., Peng, F., Yan, R., Tang, X., Yuan, J., & Deng, R. (2025). An uncertainty quantification and accuracy enhancement method for deep regression prediction scenarios. *Mechanical Systems and Signal Processing*, *227*, 112394. https://doi.org/10.1016/j.ymssp.2025.112394

Zhao, F., Liu, Y., Liu, H., & Fan, J. (2022). Broad learning approach to surrogate-assisted multi-objective evolutionary fuzzy clustering algorithm based on reference points for color image segmentation. *Expert Systems with Applications*, *200*, 117015. https://doi.org/10.1016/j.eswa.2022.117015

Zhao, H., & Lu, X. (2024). Broad learning system based on maximum multi-kernel correntropy criterion. *Neural Networks*, *179*, 106521. https://doi.org/10.1016/j.neunet.2024.106521

Zhao, H., Lu, X., & Philip Chen, C. L. (2025). Generalized maximum correntropy broad learning system with robust m-estimator. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, *55*(1), 228–237. https://doi.org/10.1109/TSMC.2024.3462801

Zheng, Y., Chen, B., Wang, S., & Wang, W. (2021). Broad learning system based on maximum correntropy criterion. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(7), 3083–3097. https://doi.org/10.1109/TNNLS.2020.3009417

Zheng, Y., Wang, S., & Chen, B. (2023). Multikernel correntropy based robust least squares one-class support vector machine. *Neurocomputing*, *545*, 126324. https://doi.org/10.1016/j.neucom.2023.126324

Zheng, Y., Wang, S., & Chen, B. (2024). Generalized multikernel correntropy based broad learning system for robust regression. *Information Sciences*, *678*, 121026. https://doi.org/10.1016/j.ins.2024.121026

Zhou, P., Lv, Y., Wang, H., & Chai, T. (2017). Data-driven robust RVFLNs modeling of a blast furnace iron-making process using cauchy distribution weighted m-estimation. *IEEE Transactions on Industrial Electronics*, *64*(9), 7141–7151. https://doi.org/10.1109/TIE.2017.2686369